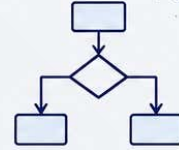


1010101010
0101010101
1010101010
0101010101



Infographics



Visual Learning

Better understanding through visual content.



Simplified Concepts

Complex topics explained in simple and visual form.



Quick Revision

Easy to revise important points and diagrams.

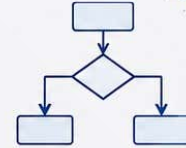


Exam Ready

Helps in faster revision and better exam preparation.



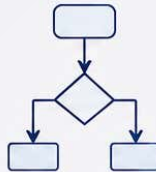
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



1. Exception Handling

- An **exception** is an abnormal situation that occurs during program execution and disrupts the normal flow.
- Exception handling is a mechanism to handle runtime errors and maintain the normal flow of the program.
- It avoids abnormal termination of the program and provides meaningful messages to the user.



Exception

occurs during runtime

3. Syntax: try - catch - finally

```
try {
    // code that may throw an exception
} catch (ExceptionType e) {
    // handling code
} finally {
    // executed always (whether exception occurs or not)
}
```

try : code that may throw exception

catch : handles the exception

finally : always executes

4. Example

```
public class Demo {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int c = a / b; // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Program Completed");
        }
    }
}
```

Output: Cannot divide by zero
Program Completed

6. Nested Try Statements

```
try {
    try {
        int a = 10 / 0;
    } catch (ArithmeticException e) {
        System.out.println("Inner try-catch: " + e);
    }
} catch (Exception e) {
    System.out.println("Outer try-catch: " + e);
}
```

2. Types of Exceptions

Exceptions

Checked Exceptions (Compile-time)

- Checked by compiler.
- Must be handled using try-catch.
- Examples: IOException, SQLException, ClassNotFoundException

Unchecked Exceptions (Runtime)

- Occurs during runtime.
- Not checked by compiler.
- Occurs due to logical errors.
- Examples: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException

5. Multiple Catch Blocks

```
try {
    int[] arr = {1,2,3};
    System.out.println(arr[5]);
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Error");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array Index Error");
} catch (Exception e) {
    System.out.println("General Exception");
}
```

8. Multithreading

- Multithreading means execution of multiple threads concurrently within a single program.
- Each thread is a separate path of execution.
- It allows better CPU utilization and improved performance.
- In Java, multithreading is achieved by using Thread class or Runnable interface.

Advantages

- Better CPU utilization
- Improved performance
- Responsive user interface
- Resource sharing

Applications

- Games
- Multimedia applications
- Web servers
- Real-time systems
- Banking applications

9. Thread in Java

- A thread is a lightweight sub-process.
- Every Java application has at least one thread (main thread).
- Threads share the same memory space.

10. Ways to Create a Thread

1. By extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
```

2. By implementing Runnable interface

```
class MyThread implements Runnable {
    public void run() {
        System.out.println("Thread running");
    }
}
```

11. Example: Creating and Running a Thread (Thread class)

```
class MyThread extends Thread {
    public void run() {
        for(int i=1; i<=5; i++) {
            System.out.println("Child Thread: " + i);
        }
    }
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts a new thread
        for(int i=1; i<=5; i++) {
            System.out.println("Main Thread: " + i);
        }
    }
}
```

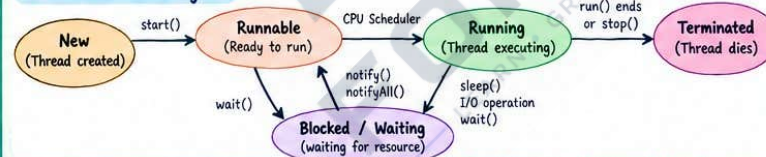
Output (order may vary):

```
Child Thread: 1
Main Thread: 1
Child Thread: 2
Main Thread: 2
Child Thread: 3
Main Thread: 3
Child Thread: 4
Main Thread: 4
Child Thread: 5
Main Thread: 5
```

12. Example: Using Runnable

```
class MyThread implements Runnable {
    public void run() {
        for(int i=1; i<=5; i++) {
            System.out.println("Thread: " + i);
        }
    }
    public static void main(String[] args) {
        MyThread obj = new MyThread();
        Thread t = new Thread(obj);
        t.start();
    }
}
```

13. Thread Life Cycle



14. Important Thread Methods

Method	Description
start()	Starts a new thread and calls run().
run()	Contains the code to be executed by the thread.
sleep(long ms)	Causes the thread to sleep for specified time.
yield()	Gives a chance for other threads to execute.
join()	Waits for another thread to complete.
isAlive()	Returns true if thread is still running.
currentThread()	Returns the current thread object.

15. Thread Priorities

- Each thread has a priority (1 to 10).
- Default priority is 5 (Thread.NORM_PRIORITY).
- Higher priority thread has more chance to execute.
- Methods:
 - setPriority(int p)
 - getPriority()

Priority Constants

```
Thread.MIN_PRIORITY = 1
Thread.NORM_PRIORITY = 5
Thread.MAX_PRIORITY = 10
```

7. throw and throws

throw

- Used to explicitly throw an exception.
- Followed by an object of Exception class.
- Generally used inside a method.

```
throw new
ArithmeticException
("Invalid Operation");
```

throws

- Used in method signature.
- Indicates that the method may throw an exception.
- Used for checked exceptions.

```
public void show()
throws IOException {
    // method code
}
```

1. What is an Exception ?

- An exception is an **abnormal situation** that occurs during the execution of a program and **disrupts the normal flow** of the program.
- It is an event, which occurs at **runtime**, that prevents the program from executing further.
- In **Java**, an exception is represented as an **object of a class**.



occurs at runtime

2. Need of Exception Handling

- It helps to handle **runtime errors gracefully** without terminating the program.
- It provides meaningful error messages to the user.
- It maintains the normal flow of the program.
- It improves the reliability and robustness of the program.
- It separates error-handling code from normal code.

3. How an Exception Occurs ?

- When the program executes, if an abnormal condition occurs, the Java Runtime System creates an **exception object**.
- This object contains information about the error.
- The object is then **thrown** to the JVM.
- If it is not handled, the program terminates abnormally and a default error message is displayed.

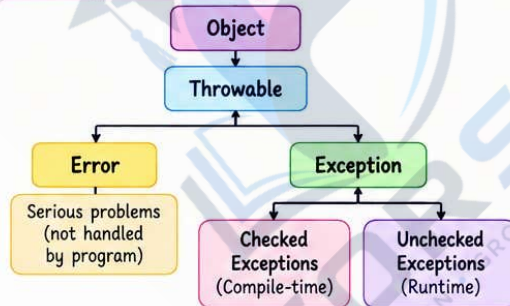
4. Example of Exception Situation

```
public class Example {
    public static void main(String[] args) {
        int a = 10, b = 0;
        int c = a / b; // Causes exception
        System.out.println("Result: " + c);
    }
}
```

Output :

Exception in thread "main"
java.lang.ArithmeticException: / by zero
at Example.main(Example.java:4)

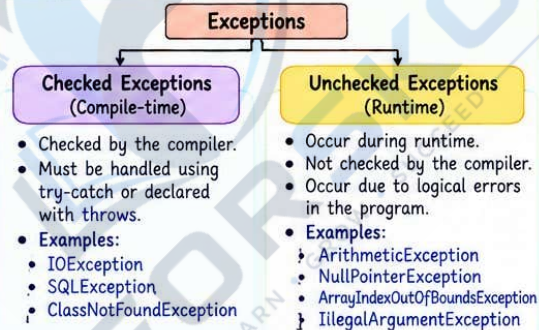
5. Exception Hierarchy



6. Key Points

- An exception is represented by an object of class **Throwable** or its subclasses.
- When an exception occurs, the normal flow of the program is disrupted.
- Java provides exception handling mechanisms like **try, catch, throw, throws and finally**.
- Exceptions can be handled using try-catch blocks.
- If an exception is not handled, the program terminates and JVM prints the error message.

7. Types of Exceptions



8. Real Life Analogy



While driving, if a stone suddenly comes on the road, you slow down or change direction instead of stopping the journey completely.

Similarly, in a program, an exception occurs, but we **handle it** to continue normal execution.

9. Advantages

- ✓ Prevents abnormal program termination.
- ✓ Provides meaningful error messages.
- ✓ Improves the reliability and robustness of the program.
- ✓ Helps in separating error-handling code from normal code.
- ✓ Maintains the normal flow of the program.
- ✓ Makes the program more user-friendly.

10. Common Scenarios Causing Exceptions

- Division by zero (ArithmeticException)
- Accessing an invalid array index (ArrayIndexOutOfBoundsException)
- Accessing an element in a null object (NullPointerException)
- Opening a file that does not exist (FileNotFoundException)
- Invalid type casting (ClassCastException)
- Invalid input (NumberFormatException)

These situations cause exceptions at runtime.

1. Introduction

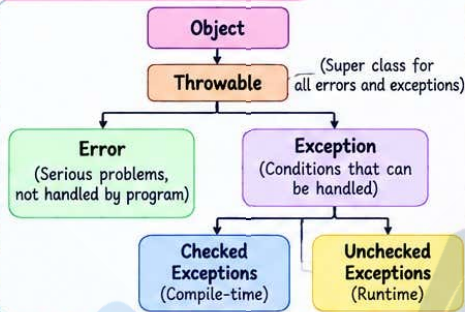
- An exception is an **abnormal condition** that occurs during the execution of a program and **disrupts the normal flow of the program**.
- Java provides exception handling mechanisms to handle these situations gracefully.
- Exceptions are represented as objects of classes that are subclasses of the **Throwable class**.



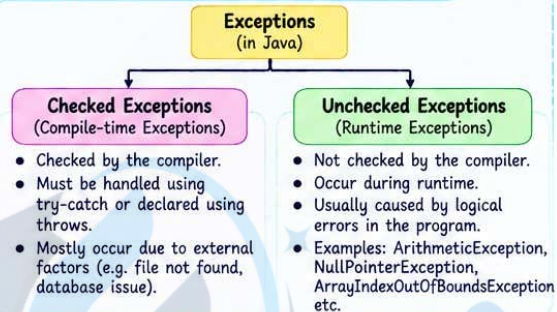
Exception

occurs at runtime

2. Base Class Hierarchy



3. Types of Exceptions (Classification)



4. Checked Exceptions

- Also called **compile-time** exceptions.
- Must be either handled using try-catch or declared using throws.
- These exceptions are checked by the Java compiler.
- They usually occur due to external factors such as file handling, network operations, or database connectivity.

Examples

- IOException
- SQLException
- ClassNotFoundException
- FileNotFoundException
- InterruptedException

Example

```

import java.io.*;
public class Example1 {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("file.txt");
            System.out.println("File opened");
            fr.close();
        } catch (IOException e) {
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
  
```

Output (if file not present)

```

File not found: file.txt
(java.io.FileNotFoundException)
  
```

5. Unchecked Exceptions

- Also called **runtime** exceptions.
- Not checked by the compiler.
- These occur due to logical errors in the program and generally cannot be predicted.
- They can be handled using try-catch, but it is not mandatory.

Examples

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException
- ClassCastException
- IllegalArgumentException

Example

```

public class Example2 {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // runtime error
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " + e.getMessage());
        }
    }
}
  
```

Output

```

Cannot divide by zero: / by zero
(java.lang.ArithmeticException)
  
```

6. Errors vs Exceptions

Basic	Error	Exception
Meaning	Serious problems in the system (not handled by program)	Abnormal conditions that can be handled
Checked by Compiler	No	Yes (checked or unchecked)
Handling	Generally not handled	Can be handled using try-catch
Examples	OutOfMemoryError, StackOverflowError	IOException, ArithmeticException, NullPointerException

7. Common Runtime Exceptions

Exception	Description
ArithmeticException	Occurs when dividing by zero.
NullPointerException	Occurs when using a null object.
ArrayIndexOutOfBoundsException	Occurs when accessing an invalid array index.
NumberFormatException	Occurs when converting an invalid string to a number.
ClassCastException	Occurs when an invalid type casting is performed.
IllegalArgumentException	Occurs when an illegal argument is passed to a method.

8. Key Points

- ✓ All exceptions in Java are subclasses of Throwable.
- ✓ Checked exceptions are handled at compile time.
- ✓ Unchecked exceptions occur at runtime.
- ✓ Errors are serious problems and are not handled by applications.
- ✓ Exception handling improves the robustness and reliability of a program.
- ✓ Common unchecked exceptions occur due to programming/logic errors.

1. Introduction

- try**, **catch** and **finally** are the blocks used in Java for exception handling.
- They help to handle runtime errors gracefully without terminating the program.
- try** block contains the code that may throw an exception.
- catch** block handles the exception.
- finally** block contains the code that **always executes**, whether an exception occurs or not (used for cleanup activities).

2. Syntax

```
try {
    // code that may throw an exception
}
catch (ExceptionType e) {
    // handle the exception
}
finally {
    // code that always executes
}
```

try block

Contains the code that may throw an exception.

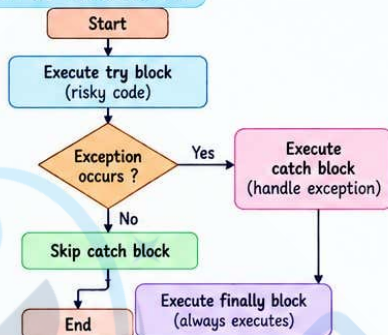
catch block

Catches and handles the exception of the specified type.

finally block

Always executes, whether an exception occurs or not.

3. Flow of Execution



4. Example 1: Exception Occurs

```
public class Demo1 {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // risky code
            System.out.println("Result: " + a);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: "
                + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

Cannot divide by zero: / by zero
Finally block executed.

5. Example 2: No Exception Occurs

```
public class Demo2 {
    public static void main(String[] args) {
        try {
            int a = 10 / 2;
            System.out.println("Result: " + a);
        } catch (ArithmeticException e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

Result: 5
Finally block executed.

6. Example 3: Multiple Catch Blocks with Finally

```
public class Demo3 {
    public static void main(String[] args) {
        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("General Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block always executes.");
        }
    }
}
```

Output:

Array Index Exception: Index 5 out of bounds for length 3
Finally block always executes.

7. Key Points

- try block must be followed by at least one catch or finally block.
- You can have multiple catch blocks to handle different types of exceptions.
- finally block is optional, but recommended for cleanup activities (like closing files, database connections, etc.).
- finally block always executes, whether an exception occurs or not.
- If an exception occurs and is not handled by any catch block, it is propagated to the JVM.

8. Important Notes

- The code after try-catch-finally will execute only if the exception is handled.
- finally block is useful for releasing resources (e.g., closing files, database connections).
- From Java 7 onwards, try-with-resources is introduced for automatic resource management.
- We can also use multiple catch blocks and a general catch (Exception) at the end.

9. Real Life Analogy



Imagine you are solving a question paper.

- try block** → You attempt all the questions.
- catch block** → If you cannot solve a question, you handle it (skip or write something else).
- finally block** → At the end, you always submit the paper (this will happen whether you solve all questions or not).

1. What are Multiple Catch blocks ?

- In Java, we can use **multiple catch blocks** to handle **different types of exceptions** separately.
- If a try block may throw more than one type of exception, then multiple catch blocks are used.
- Each catch block handles a **specific type of exception**.
- The catch blocks are checked in sequence, so the **first matching catch block is executed**.

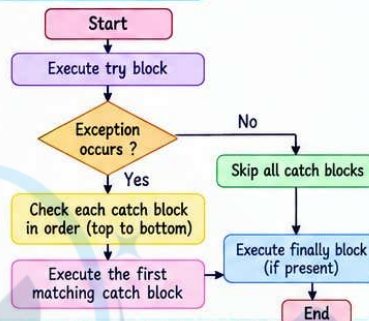
2. Syntax

```
try
    // code that may throw exceptions
}
catch (ExceptionType1 e1) {
    // handle exception 1
}
catch (ExceptionType2 e2) {
    // handle exception 2
}
catch (ExceptionType3 e3) {
    // handle exception 3
}
finally {
    // optional: code always executes
}
```

3. Rules for Multiple Catch blocks

- Each catch block must handle a **different type** of exception.
- The catch blocks must be arranged from **specific to general**.
- Subclass exceptions should be caught before superclass exceptions.
- If a superclass exception (e.g., Exception) is placed before a subclass exception (e.g., ArithmeticException), it will give a **compile-time error** (unreachable catch block).
- Only the **first matching catch block** is executed.
- We can also use multiple catch blocks along with **finally**.

4. Flow of Execution



5. Example 1: Handling Different Exceptions

```
public class Demo1 {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int c = a / b; // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: "
                + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception: "
                + e.getMessage());
        } catch (Exception e) {
            System.out.println("General Exception: "
                + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

Arithmetic
Exception:
/ by zero

Finally block
executed.

Explanation:

- ArithmeticException occurs because we divide by zero.
- The first matching catch block is executed, so other catch blocks are skipped.
- Finally block always executes.

6. Example 2: Array Index Exception

```
public class Demo2 {
    public static void main(String[] args) {
        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: "
                + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception: "
                + e.getMessage());
        } catch (Exception e) {
            System.out.println("General Exception: "
                + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

Array Index Exception: Index 5 out of bounds for length 3
Finally block executed.

7. Example 3: File Not Found Exception

```
import java.io.*;
public class Demo3 {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("data.txt");
            System.out.println("File opened");
            fr.close();
        } catch (FileNotFoundException e) {
            System.out.println("File Not Found: "
                + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO Exception: "
                + e.getMessage());
        } catch (Exception e) {
            System.out.println("General Exception: "
                + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

File Not Found:
data.txt
(No such file
or directory)

Finally block
executed.

8. Important Points

- Multiple catch blocks are used to handle multiple types of exceptions.
- Catch blocks are checked in top-to-bottom order.
- The first matching catch block is executed.
- Subclass exceptions must be caught before superclass exceptions.
- Finally block (if present) always executes.
- If no catch block matches, the exception is propagated to the JVM.

9. Commonly Used Exceptions in Catch Blocks

Exception	Description
ArithmeticException	Occurs when dividing by zero.
ArrayIndexOutOfBoundsException	Occurs when accessing an invalid array index.
NumberFormatException	Occurs when converting an invalid string to a number.
FileNotFoundException	Occurs when a file is not found.
IOException	Occurs during input/output operations.
NullPointerException	Occurs when accessing a null object.
Exception	Superclass of all exceptions (general).

10. Key Takeaways

- Use multiple catch blocks to handle different exceptions separately.
- Always place specific exceptions before general exceptions.
- If a general exception (Exception) is placed first, other catch blocks become unreachable and cause a compile-time error.
- Only one catch block executes (the first matching one).
- Finally block is optional but recommended for cleanup activities (e.g., closing files, database connections, etc.).

1. What are Nested Try Statements ?

- A nested try statement means a **try block inside another try block**.
- It is used when a part of the code inside a try block can also throw another exception.
- The **inner try-catch** handles exceptions of the inner block, while the **outer try-catch** handles exceptions of the outer block.
- It helps to handle multiple exceptions at different levels in a program.

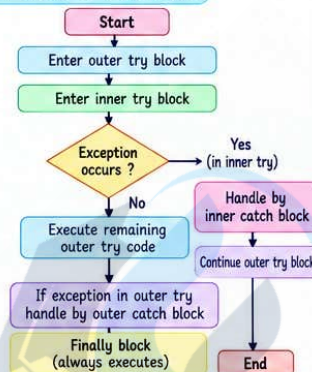
2. Syntax of Nested Try

```
try {
    // outer try block
    try {
        // inner try block
        // statements that may throw exception
    } catch (ExceptionType1 e1) {
        // handle inner exception
    }
    // other statements
} catch (ExceptionType2 e2) {
    // handle outer exception
} finally {
    // optional: executed always
}
```

Outer try block

Inner try block

3. Flow of Execution



4. Important Points

- ✓ A try block can be nested inside another try block.
- ✓ Inner try block has its own catch block(s).
- ✓ Outer try block can have its own catch block(s).
- ✓ If an exception is not handled by inner catch, it propagates to the outer catch block.
- ✓ **finally** block (if present) always executes, whether exception occurs or not.
- ✓ Nested try is useful when different parts of a program may throw different types of exceptions.

5. Example 1: Nested Try with Arithmetic Exception

```
public class NestedTryDemo1 {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block starts");
            int a = 10;
            try {
                System.out.println("Inner try block!");
                int b = a / 0; // ArithmeticException
            } catch (ArithmeticException e1) {
                System.out.println("Inner catch: " + e1.getMessage());
            }
            System.out.println("Outer try block continues");
        } catch (Exception e2) {
            System.out.println("Outer catch: " + e2.getMessage());
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output:

Outer try block starts
 Inner try block
 Inner catch: / by zero
 Outer try block continues
 Finally block executed

Explanation:

- ArithmeticException occurs in inner try block.
- It is handled by the inner catch block.
- Outer catch block is not executed because the exception is already handled.
- Finally block executes at the end.

7. Example 3: Multiple Catch in Nested Try

```
public class NestedTryDemo3 {
    public static void main(String[] args) {
        try {
            String str = "Java";
            try {
                System.out.println(str.charAt(10)); // StringIndexOutOfBoundsException
            } catch (StringIndexOutOfBoundsException e1) {
                System.out.println("Inner catch: " + e1.getMessage());
            } catch (Exception e) {
                System.out.println("Inner catch (General): " + e.getMessage());
            }
        } catch (Exception e2) {
            System.out.println("Outer catch: " + e2.getMessage());
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output:

Inner catch: String index out of range: 10
 Finally block executed

Explanation:

- StringIndexOutOfBoundsException is handled by inner catch block.
- Outer catch is not executed.
- Finally block executes at the end.

6. Example 2: Exception not Handled in Inner Try

```
public class NestedTryDemo2 {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                int arr[] = {1, 2, 3};
                System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
            } catch (ArithmeticException e1) { // wrong catch type
                System.out.println("Inner catch: " + e1.getMessage());
            }
        } catch (ArrayIndexOutOfBoundsException e2) {
            System.out.println("Outer catch: " + e2.getMessage());
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output:

Outer try block
 Outer catch: Index 5 out of bounds for length 3
 Finally block executed

Explanation:

- ArrayIndexOutOfBoundsException occurs in inner try block.
- It is not handled by inner catch (since catch type is different).
- So, it propagates to the outer catch block.
- Finally block executes at the end.

8. Use Cases of Nested Try

- When a part of the code inside a try block can throw a different exception.
- When you want to handle some exceptions locally (inner try) and more general exceptions globally (outer try).
- Useful in complex applications (e.g., file handling, database operations, network programming).
- Helps in better error handling and program stability.

9. Key Takeaways

- ✓ Nested try allows a try block inside another try block.
- ✓ Inner and outer try blocks have separate catch blocks.
- ✓ If an exception is not handled in inner try, it is propagated to the outer catch.
- ✓ **finally** block always executes, whether an exception occurs or not.
- ✓ It improves exception handling in complex programs.

1. Introduction

- In Java, **throw** and **throws** are keywords used for exception handling.
- Both are used to deal with exceptions, but they have different purposes.
- throw** is used to **explicitly throw** an exception object.
- throws** is used to **declare** that a method may throw one or more exceptions.

2. throw keyword

- The **throw** keyword is used to **explicitly throw an exception object**.
- It is used **inside a method or block**.
- It is followed by an instance of **Exception class** or its subclass.
- It is used when we want to throw a **custom exception** based on a condition.

Syntax :

```
throw new ExceptionClass("message");
```

3. throws keyword

- The **throws** keyword is used to **declare that a method may throw one or more exceptions**.
- It is used in the **method signature (declaration)**.
- It **does not throw** the exception, it just informs the caller.
- It is followed by one or more exception classes, separated by commas.

Syntax :

```
returnType methodName() throws  
ExceptionClass1, ExceptionClass2 { ... }
```

4. Key Differences

Basis	throw	throws
Purpose	Used to explicitly throw an exception object.	Used to declare that a method may throw exceptions.
Used in	Inside a method or block.	In method signature.
Followed by	An instance of Exception class (new object).	Exception class name(s).
Can we throw multiple?	Only one exception at a time.	Multiple exceptions can be declared (separated by commas).
Effect	Actually throws the exception.	Just declares, does not throw.

5. Example: Using throw

```
public class ThrowExample {  
    public static void main(String[] args) {  
        int age = 15;  
        if (age < 18) {  
            throw new ArithmeticException(  
                "Age must be 18 or above");  
        } else {  
            System.out.println("Eligible to vote");  
        }  
    }  
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException:
Age must be 18 or above

6. Example: Using throws

```
import java.io.*;  
public class ThrowsExample {  
    static void readFile() throws IOException {  
        FileReader fr = new FileReader("data.txt");  
        System.out.println("File opened");  
        fr.close();  
    }  
    public static void main(String[] args) {  
        try {  
            readFile();  
        } catch (IOException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

Output (if file not present):

Error: data.txt (No such file or directory)

7. Example: throws with Multiple Exceptions

```
public class MultipleThrows {  
    static void method() throws ArithmeticException,  
        NullPointerException, ArrayIndexOutOfBoundsException {  
        int[] arr = null;  
        System.out.println(arr[0]);  
    }  
    public static void main(String[] args) {  
        try {  
            method();  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception");  
        } catch (NullPointerException e) {  
            System.out.println("Null Pointer Exception");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Exception");  
        }  
    }  
}
```

Output:

Null Pointer
Exception

Explanation:

The method() may throw multiple exceptions, so they are declared using throws and handled using multiple catch blocks.

8. Real Life Analogy

throw → Throwing an exception



You are actually throwing the problem (like a ball) to stop the program and show an error.

throws → Informing about possibility



You are just informing others that this method might throw an exception (a warning).

9. Important Points

- throw** is used to explicitly throw an exception.
- throws** is used to declare exceptions in a method.
- throw** is followed by an object, **throws** is followed by class name(s).
- throw** can be used anywhere inside a method, **throws** in method signature.
- Only one exception can be thrown at a time using **throw**.
- Multiple exceptions can be declared using **throws**.
- Both are used with exception handling (try-catch).
- If an exception is thrown using **throw** and not handled, the program terminates.
- throws** is mostly used for **checked exceptions**.

10. Common Exceptions Used with throw/throws

Exception	Description
ArithmeticException	Occurs due to arithmetic errors (e.g., divide by zero).
NullPointerException	Occurs when accessing a null object.
ArrayIndexOutOfBoundsException	Occurs when array index is invalid.
IOException	Occurs during input/output operations.
ClassNotFoundException	Occurs when a class is not found.
SQLException	Occurs for database access errors.
IllegalArgumentException	Occurs when an illegal argument is passed.

1. Introduction to Multithreading

- Multithreading is a process of executing **multiple threads** (sub-programs) in a single program simultaneously.
- Each thread works **independently** and performs a specific task.
- Multithreading allows **better CPU utilization**, improves **performance** and **responsiveness** of an application.
- In Java, multithreading is achieved using **Thread class** and **Runnable interface**.

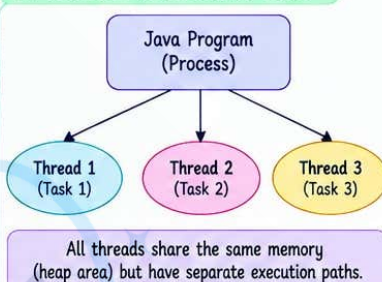
2. What is a Thread?

- A thread is the **smallest unit of execution** in a program.
- It is a **lightweight** sub-process within a process.
- Each thread has its own path of execution, but shares the **same memory space** (heap) with other threads of the same program.
- Threads enable **concurrent execution** of tasks.

3. Process vs Thread

Process	Thread
A process is an independent program in execution.	A thread is a part of a process.
Has its own memory space.	Shares the same memory space with other threads.
Heavyweight.	Lightweight.
Creation and context switching is time consuming.	Creation and context switching is faster.
Processes are isolated from each other.	Threads within a process can communicate easily.
Example: Notepad, Chrome, Word etc.	Example: Multiple threads in a web browser.

4. Thread Execution in Java

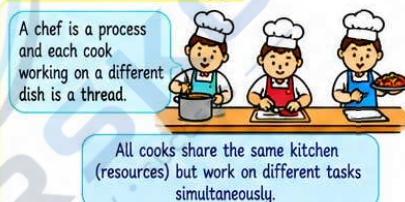


5. Advantages of Multithreading

- ✓ Better CPU utilization
- ✓ Improved performance
- ✓ Responsive applications (e.g., GUI)
- ✓ Concurrent execution of multiple tasks
- ✓ Resource sharing (memory)
- ✓ Efficient for I/O intensive tasks (e.g., file read/write, network communication)



6. Real Life Analogy



7. When to use Multithreading?

- ✓ When a program needs to perform multiple tasks at the same time.
- ✓ When we want to keep the application responsive (e.g., GUI applications).
- ✓ When tasks are independent and can run concurrently.
- ✓ Useful in file handling, database operations, network programming, games, etc.

8. Creating a Thread (Basic Concept)

1. By extending Thread class

```

class MyThread extends Thread {
    public void run() {
        System.out.println(
            "Thread is running...");
    }
}
  
```

2. By implementing Runnable interface

```

class MyThread implements Runnable {
    public void run() {
        System.out.println(
            "Thread is running...");
    }
}
  
```

9. Simple Example: Thread Basics

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts a new thread
        System.out.println("Main thread is running...");
    }
}
  
```

10. Output

```

Main thread is running...
Hello from a thread!
  
```

Note:

- Both main thread and new thread execute concurrently.
- The order of output may vary every time.
- start()** method is used to begin the execution of a new thread.

11. Key Points

- ✓ A thread is the smallest unit of execution.
- ✓ Multithreading allows multiple threads to run concurrently.
- ✓ In Java, threads can be created by extending **Thread** class or implementing **Runnable** interface.
- ✓ Each thread executes the **run()** method.
- ✓ **start()** method starts a new thread.
- ✓ Threads share the same memory space.
- ✓ It improves performance and responsiveness of the program.

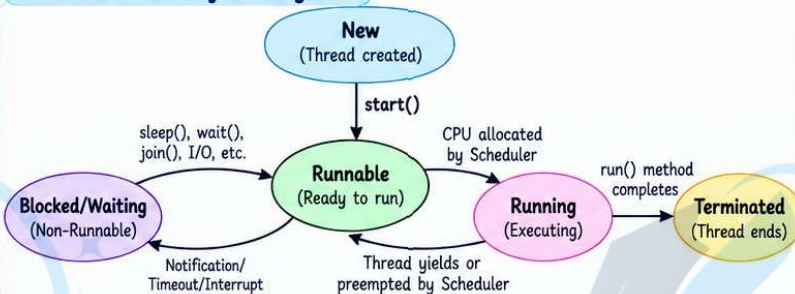
12. Common Methods of Thread Class

Method	Description
start()	Starts a new thread (calls run() method).
run()	Contains the code to be executed by the thread.
sleep(long millis)	Pauses the thread for specified time.
join()	Waits for a thread to die.
isAlive()	Returns true if thread is still running.
getName()	Returns the name of the thread.
setName(String)	Sets the name of the thread.
getPriority()	Returns the priority of the thread.
setPriority(int)	Sets the priority of the thread.
currentThread()	Returns the current executing thread.

1. What is Thread Life Cycle ?

- The thread life cycle shows the different states through which a thread passes from **creation** to **termination**.
- A thread changes its state depending on the actions performed by the program and the JVM.
- Each state represents a stage of thread execution.
- The life cycle is managed by the **JVM (Java Virtual Machine)**.

2. Thread Life Cycle Diagram



3. States of a Thread

1 New	Thread object is created but start() is not yet called.
2 Runnable	Thread is ready to run and waiting for CPU.
3 Running	Thread is currently executing the run() method.
4 Blocked / Waiting	Thread is temporarily not runnable (waiting for a resource, notification or timeout).
5 Terminated	Thread has completed execution and run() method has finished.

4. State Transitions and Methods

From State	To State	Method / Condition	Description
New	Runnable	start()	Thread becomes ready to run.
Runnable	Running	Thread Scheduler allocates CPU	Thread starts executing run().
Running	Runnable	yield() or time slice over	Thread gives up CPU and goes back to runnable state.
Running	Blocked/Waiting	sleep(), wait(), join(), I/O operations	Thread stops execution temporarily.
Blocked/Waiting	Runnable	notify(), notifyAll(), interrupt(), timeout or I/O completion	Thread becomes runnable again.
Running	Terminated	run() method ends or stop() (deprecated)	Thread completes execution.

5. Example Program (Demonstrating States)

```

class LifeCycleDemo extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);
            try {
                Thread.sleep(500); // causes Waiting state
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        LifeCycleDemo t = new LifeCycleDemo();
        System.out.println("State after creation: " + t.getState()); // NEW
        t.start();
        System.out.println("State after start(): " + t.getState()); // Runnable/Running
        Thread.sleep(200); // give time to run
        System.out.println("State during execution: " + t.getState());
        t.join(); // main thread waits for t to finish
        System.out.println("State after completion: " + t.getState()); // TERMINATED
    }
}
  
```

Sample Output:

```

State after creation: NEW
Child Thread: 1
Child Thread: 2
State during execution: TIMED_WAITING
Child Thread: 3
Child Thread: 4
Child Thread: 5
State after completion: TERMINATED
  
```

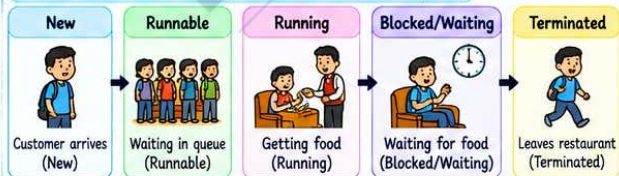
6. Methods Affecting Thread States

Method	Effect on Life Cycle
start()	Moves thread from New to Runnable.
run()	Contains code to be executed (Running state).
sleep(long millis)	Moves thread to Timed Waiting state.
wait()	Moves thread to Waiting state (requires synchronized).
notify() / notifyAll()	Wakes up waiting thread and moves it to Runnable.
yield()	Gives up CPU, moves from Running to Runnable.
join()	Causes calling thread to wait until target thread terminates (Waiting state).
interrupt()	Interrupts a sleeping or waiting thread.
isAlive()	Returns true if thread is in Runnable or Running state.
getState()	Returns the current state of the thread.

7. Important Points

- A thread always starts in **New state**.
- Thread must call **start()** to move to **Runnable** state.
- The thread scheduler decides which thread gets CPU.
- A thread can move between **Runnable** and **Running** many times.
- sleep()**, **wait()**, and **join()** put the thread into **blocked/waiting** state.
- notify()** or **timeout** moves the thread back to **runnable** state.
- Once **run()** method completes, the thread enters **terminated state** and cannot be started again.

8. Real Life Analogy (Restaurant Example)



1. What is Multithreading ?

- Multithreading is a feature of Java that allows **multiple threads** (sub-programs) to execute **concurrently** within a **single program**.
- Each thread performs a separate task, but all threads share the **same memory space** (heap) and resources.
- It helps in **better CPU utilization**, improved **performance** and **responsiveness**.
- In Java, multithreading is achieved using the **Thread class** and **Runnable interface**.

2. Concept / Idea of Multithreading

- A program usually has a **single thread** of execution (main thread) which executes instructions **sequentially**.
- Multithreading breaks a program into **multiple smaller threads**, each performing a different task.
- All threads run independently but share the **same process, memory and resources**.
- Threads can execute **concurrently** (actual parallel execution depends on CPU and OS).
- Useful for applications like GUI, games, web servers, database applications, etc.

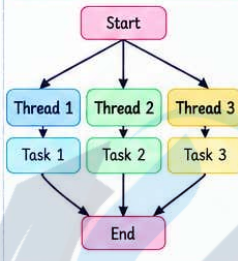
3. Single Thread vs Multithreaded

Single Threaded Program



Tasks are executed one by one (**sequentially**).

Multithreaded Program



Multiple tasks are executed **concurrently** (independently).

4. Multithreading in Real Life



Web Browser

Multiple tabs run simultaneously



Media Player

Play video and audio at the same time



Printer

Print multiple documents in background



Mobile Phone

Run multiple apps at the same time

5. Features / Characteristics

- Multiple threads run **concurrently**.
- Threads share the **same** memory space (heap) and resources.
- Each thread executes independently.
- Improves **CPU utilization**.
- Increases **performance and responsiveness**.
- Lightweight compared to processes.
- Useful for time-consuming or background tasks (e.g., file handling, network programming).

6. Advantages

- Better CPU utilization
- Improved performance and fast execution
- Responsive applications (e.g., GUI remains active)
- Efficient resource sharing
- Useful for multiple tasks (e.g., file read/write, network communication, games, etc.)



7. Disadvantages

- Complex to design and debug.
- Synchronization is required to avoid data inconsistency.
- Resource sharing may cause **deadlock** and **starvation**.
- More difficult error handling.
- Context switching overhead may reduce performance in some cases.

8. Simple Example of Multithreading

```

class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);
            try {
                Thread.sleep(500); // pause for a while
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public class TestMultithreading {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts a new thread
        for (int i = 1; i <= 5; i++) {
            System.out.println("Main Thread: " + i);
        }
    }
}
  
```

9. Important Points

- A thread is the **smallest unit** of execution.
- Multithreading allows multiple tasks to run in the same program.
- Threads share the same memory space.
- In Java, multithreading is implemented using **Thread class** or **Runnable interface**.
- Threads are used for concurrent execution of tasks.
- Useful in real-time and large applications.



10. Applications of Multithreading



GUI Applications

Keeps GUI responsive (e.g., window, button events)



Database Applications

Handle multiple requests simultaneously



Network Programming

Handle multiple client requests at the same time



Games

Handle multiple tasks like graphics, sound, user input

1. Ways to Create a Thread

- In Java, a thread can be created in two ways:

- By extending Thread class
- By implementing Runnable interface



Both ways create a new thread, but **Runnable interface** is recommended because Java does not support multiple inheritance.

2. Important Methods

Method	Description
start()	Starts a new thread. JVM calls run() method.
run()	Contains the code to be executed by the thread.
sleep(long millis)	Pauses the thread for specified time.
currentThread()	Returns the currently executing thread.
getName()	Returns the name of the thread.
setName(String)	Sets the name of the thread.
isAlive()	Checks if the thread is still running.
join()	Waits for the thread to complete.
yield()	Gives a chance to other threads to execute.

3. Life Cycle Flow (Creating & Running)

Create a Thread
(using Thread class or Runnable interface)

Call **start()** method

JVM internally calls
run() method

Thread executes
its task

Thread completes
and terminates

4. Creating Thread by Extending Thread Class

```
// Example: Creating and Running Thread using Thread class
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);
            try {
                Thread.sleep(500); // pause for 0.5 sec
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public static void main(String[] args) {
    MyThread t = new MyThread(); // create thread
    t.start(); // start the thread
    System.out.println("Main Thread is running...");
}
```

5. Output (Thread Class Example)

```
Main Thread is running...
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5
```

Note:

- Output order may vary each time because threads run concurrently.
- Main thread and child thread execute independently.

6. Creating Thread by Implementing Runnable Interface

```
// Example: Creating and Running Thread using Runnable interface
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public class TestRunnable {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable(); // create task
        Thread t = new Thread(r); // create thread object
        t.start(); // start the thread
        System.out.println("Main Thread is running...");
    }
}
```

7. Output (Runnable Example)

```
Main Thread is running...
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5
```

Key Point:

- Here we pass the Runnable object to the Thread class.
- Thread executes the run() method of the Runnable object.

8. Comparison: Thread Class vs Runnable Interface

Basis	Thread class (extends)	Runnable interface (implements)
Approach	Inheritance	Interface
Method	Override run() method	Implement run() method
Usage	Create class that extends Thread	Create class that implements Runnable and pass to Thread
Multiple inheritance	Not possible (Java does not support multiple inheritance)	Possible (can implement multiple interfaces)
Recommended	Used for simple programs	Preferred in real applications

9. Running (Multiple Threads)

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread-" + i);
            try { Thread.sleep(500); } catch (Exception e) {}
        }
    }
}

public static void main(String[] args) {
    MyThread t1 = new MyThread();
    MyThread t2 = new MyThread();
    t1.setName("Thread-1");
    t2.setName("Thread-2");
    t1.start();
    t2.start();
}
```

10. Output (Multiple Threads)

```
Thread-1 : 1
Thread-2 : 1
Thread-1 : 2
Thread-2 : 2
Thread-1 : 3
Thread-2 : 3
Thread-1 : 4
Thread-2 : 4
Thread-1 : 5
Thread-2 : 5
```

Note:

- The output is interleaved because both threads run concurrently.
- Order may change every time you run the program.



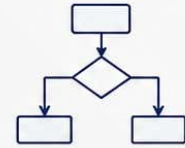
Thread-1

Thread-2

11. Important Points

- A new thread is created using Thread class or Runnable interface.
- Use **start()** method to run a thread.
- JVM calls the **run()** method internally.
- Threads execute concurrently (actual parallel execution depends on CPU and OS).
- Each thread has its own path of execution.
- Use **setName()** and **getName()** to identify threads.
- Use **join()** if one thread has to wait for another.
- Runnable interface is preferred for real applications.
- Output order may vary.

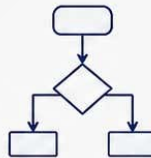
1010101010
0101010101
1010101010
0101010101



Unit - II



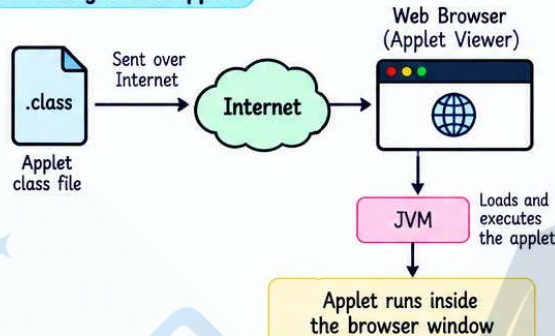
10110
01001
10101



1. What is an Applet ?

- An applet is a **small Java program** that runs inside a **web browser**.
- It is a part of Java's **AWT (Abstract Window Toolkit)**.
- Applet does not have a **main()** method. Instead, it runs **automatically** when it is loaded by the browser or **applet viewer**.
- It is **platform independent**, secure and suitable for creating **dynamic web content** like animations, games, charts, etc.

2. Working of an Applet



3. Features of Applet

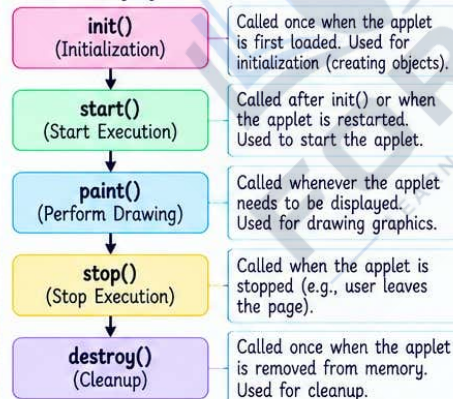
- ✓ **Small program** that runs inside a **browser**.
- ✓ Does not require **main()** **method**.
- ✓ Executed **automatically** by the browser or applet viewer.
- ✓ **Platform independent** (Write Once, Run Anywhere).
- ✓ Provides **security** (runs in a restricted environment).
- ✓ Useful for creating **interactive** and **dynamic web pages** (GUI, animation, graphics, games).

4. Characteristics of Applet

- Applets are usually **embedded** in an **HTML page**.
- They are lightweight and **network-friendly**.
- They **cannot read or write files** on the local system due to **security restrictions**.
- Communication with the user is done through GUI components.

5. Life Cycle of an Applet

An applet goes through a series of methods called the **Applet life cycle** which are called automatically by the browser.



6. Basic Structure of an Applet Program

```

import java.applet.Applet; // Import Applet class
import java.awt.Graphics; // Import Graphics class

public class MyApplet extends Applet {

    public void init() {
        // initialization code
    }

    public void start() {
        // start execution code
    }

    public void paint(Graphics g) {
        g.drawString("Hello Applet!", 50, 50);
    }

    public void stop() {
        // stop execution code
    }

    public void destroy() {
        // cleanup code
    }
}
  
```

Import required packages: Applet and Graphics.
 Class extends **Applet** class.
 Called once for initialization.
 Called when applet starts.
 Used for drawing output on the applet window. (Graphics object is used for drawing text, shapes, etc.)
 Called when applet is stopped.
 Called when applet is destroyed.

7. Running an Applet

Using Applet Viewer
(Command Prompt)

1. Compile the applet:
`javac MyApplet.java`
2. Run using **appletviewer**:
`appletviewer MyApplet.java`

Using Web Browser
(HTML File)

1. Create an HTML file (e.g., test.html) and embed the applet using **<applet>** tag.
2. Open the HTML file in a web browser.

8. Simple Example: Hello Applet

```

import java.applet.Applet;
import java.awt.Graphics;

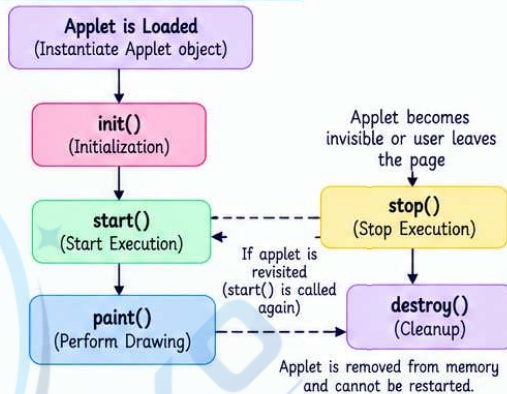
public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello Applet!", 50, 50);
    }
}
  
```

Compile: `javac HelloApplet.java`
 Run: `appletviewer HelloApplet.java`

1. What is Applet Life Cycle ?

- The applet life cycle is the **sequence of method calls** that are automatically invoked by the browser or applet viewer to manage the execution of an applet.
- These methods are defined in the **java.applet.Applet** class.
- The applet life cycle controls the creation, initialization, execution, **painting, stopping and destruction** of an applet.
- It is handled **automatically** by the **browser or applet viewer**, not by the programmer.

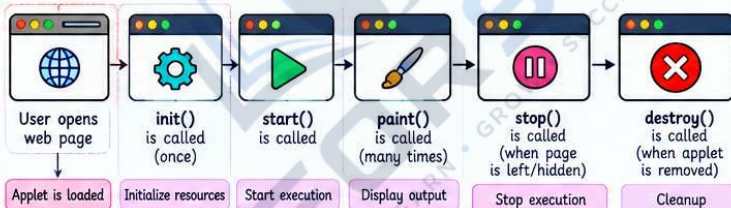
2. Applet Life Cycle Diagram



3. Explanation of Each Method

Method	Purpose	Description
init()	Initialization	- Called only once when the applet is first loaded. - Used to initialize variables, create objects, allocate resources, etc.
start()	Start Execution	- Called after init() or when the applet is restarted. - Used to start the execution or resume the applet.
paint(Graphics g)	Perform Drawing	- Called whenever the applet needs to be displayed. - Used for drawing text, shapes, graphics, etc.
stop()	Stop Execution	- Called when the applet becomes invisible (e.g., user leaves the page). - Used to stop execution and release resources such as threads.
destroy()	Cleanup	- Called only once when the applet is removed from memory. - Used to perform cleanup operations and free resources.

4. Life Cycle Flow in Real Life (Browser Scenario)



5. Example Program (Demonstrating Applet Life Cycle)

```

import java.applet.Applet;
import java.awt.*;

public class LifeCycleApplet extends Applet {
    public void init() {
        System.out.println("init() - Applet initialized");
    }
    public void start() {
        System.out.println("start() - Applet started");
    }
    public void paint(Graphics g) {
        g.setColor(Color.blue);
        g.drawString("Hello Applet!", 50, 50);
        System.out.println("paint() - Applet painted");
    }
    public void stop() {
        System.out.println("stop() - Applet stopped");
    }
    public void destroy() {
        System.out.println("destroy() - Applet destroyed");
    }
}
  
```

6. Output (Console / Browser)

```

init() - Applet initialized
start() - Applet started
paint() - Applet painted
(When page is closed or hidden)
stop() - Applet stopped
destroy() - Applet destroyed
  
```

8. Important Points / Interview Questions

- The life cycle methods are defined in the **Applet class**.
- The programmer should not call these methods explicitly; they are invoked by the browser or applet viewer.
- paint()** method is used for drawing and is called multiple times.
- stop()** and **destroy()** are used to release resources.
- If the applet is revisited, **start()** is called again but **init()** is not called again.
- The applet life cycle is similar to the life cycle of a window in GUI applications.

7. Key Points

- ✓ **init()** is called **only once**.
- ✓ **start()** is called **after init()** or when the applet is revisited.
- ✓ **paint()** is called **many times** whenever the applet needs to be displayed.
- ✓ **stop()** is called when the applet becomes invisible.
- ✓ **destroy()** is called once when the applet is removed from memory.
- ✓ The applet life cycle is managed by the **browser or applet viewer automatically**.

Java Application

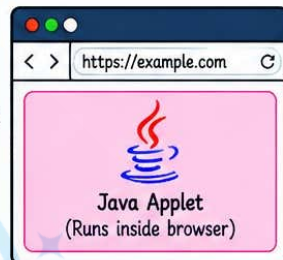
- A Java application is a **complete program** that runs **independently** on a local machine.
- It has a **main() method** as the entry point.
- It is executed through the **Java interpreter** or **JVM** from the command line or an IDE.
- It is **not embedded** in a web page.
- It can access **all system resources** (file system, network, etc.).
- It can run as a **standalone program**.



Runs independently
on local machine

Java Applet

- A Java applet is a **small program** that runs inside a **web browser** or applet viewer.
- It **does not have a main() method**. Instead, it runs automatically when loaded by the browser.
- It is **embedded** in an HTML web page using **<applet>** or **<object>** tag.
- It has **restricted access** to system resources due to security reasons.
- It requires a **browser** or **applet viewer** to run.
- It is mainly used to create **interactive web content** like animations, games, etc.



Key Differences between Java Application and Java Applet

Feature	Application	Applet
1. Definition	A complete Java program that runs independently on a local machine.	A small Java program that runs inside a web browser or applet viewer.
2. Entry Point	Has a main() method .	Does not have a main() method .
3. Execution	Executed through JVM from command line or IDE.	Executed automatically by the browser/applet viewer.
4. Embedding	Not embedded in any web page.	Embedded in an HTML page using <applet> or <object> tag.
5. System Resources	Can access all system resources (file system, network, etc.).	Has restricted access to system resources due to security.
6. Usage	Used to create standalone programs (e.g., desktop applications).	Used to create interactive web content (e.g., animations, games).
7. Security	Runs with full permissions .	Runs in a restricted environment (sandbox).
8. GUI Components	Can use all AWT/Swing components.	Usually uses AWT components and is embedded in a browser window.
9. Lifecycle Methods	Does not have lifecycle methods.	Has lifecycle methods: init() , start() , paint() , stop() , destroy() .
10. Example	 <pre>public class MyApplication { public static void main(String[] args) { System.out.println("Hello Application"); } }</pre>	 <pre>import java.applet.Applet; import java.awt.Graphics; public class MyApplet extends Applet { public void paint(Graphics g) { g.drawString("Hello Applet", 50, 50); } }</pre>

1. What is <applet> Tag ?

- The **<applet>** tag is an **HTML tag** used to **embed a Java applet** in a web page.
- It tells the **browser to load and run** the specified applet class inside the browser window (applet viewer).
- It provides parameters like **size**, **location** of class file, and additional information using attributes.
- This tag is **platform independent** and was widely used before Java Web Start and Java Plug-in.

2. Syntax of <applet> Tag

```
<applet code = "AppletClassName.class"
       width = "width"
       height = "height"
       [other attributes]>
```

```
<param name = "parameter_name" value = "value">
```

```
<!-- Applet will be displayed here -->
```

```
</applet>
```

3. General Form with All Attributes

```
<applet
  code = "AppletClassName.class"
  codebase = "URL_or_Folder"
  archive = "jarfile.jar"
  width = "width_in_pixels"
  height = "height_in_pixels"
  align = "alignment"
  hspace = "horizontal_space"
  vspace = "vertical_space"
  name = "applet_name"
  id = "applet_id"
  alt = "alternative_text"
  mayscript = "true | false" >

<param name = "parameter_name" value = "value">
Your browser does not support Java Applets.
</applet>
```

4. Output Example



The applet is displayed inside the web page using **<applet>** tag.

5. Attributes of <applet> Tag

Attribute	Description	Possible Values	Default
code	Specifies the applet class file (to be loaded).	ClassName.class	—
codebase	Specifies the location (URL or folder) of the class file.	URL or folder path	current directory
archive	Specifies the JAR file containing the applet class.	jarfile.jar	—
width	Specifies the width of the applet window.	pixels (e.g., 300)	0
height	Specifies the height of the applet window.	pixels (e.g., 200)	0
align	Specifies the alignment of the applet in the web page.	left right top middle bottom	middle
hspace	Specifies the horizontal space around the applet.	pixels (e.g., 10)	0
vspace	Specifies the vertical space around the applet.	pixels (e.g., 10)	0
name	Specifies the name of the applet.	any valid name	—
id	Specifies a unique identifier for the applet.	any valid id	—
alt	Specifies alternative text to display if applet cannot be loaded.	any text	—
mayscript	Specifies whether the applet can interact with JavaScript.	true false	false

6. Example: Simple Applet with All Attributes

```
<html>
<head><title>Applet Example</title></head>
<body>
  <h2>Example of Applet Tag with All Attributes</h2>
  <applet
    code = "HelloApplet.class"
    codebase = "."
    archive = "HelloApplet.jar"
    width = "300"
    height = "200"
    align = "center"
    hspace = "10"
    vspace = "10"
    name = "MyApplet"
    id = "ap1"
    alt = "Applet Not Found"
    mayscript = "true">

    <param name = "bgcolor" value = "yellow">
    <param name = "message" value = "Hello from Applet">

    Your browser does not support Java Applets.

  </applet>
</body>
</html>
```

7. Multiple <param> Tags

- The **<param>** tag is used to pass parameters to the applet.
- It consists of two attributes:
 - name** → name of the parameter
 - value** → value of the parameter
- Multiple **<param>** tags can be used inside the **<applet>** tag.

Example:

```
<param name = "bgcolor" value = "pink">
<param name = "fontcolor" value = "blue">
<param name = "message" value = "Welcome">
```

8. Important Points

- ✓ The **<applet>** tag is used to embed a Java applet in an HTML page.
- ✓ It supports various attributes to control the appearance and behavior of the applet.
- ✓ The **code** attribute specifies the applet class file.
- ✓ The **width** and **height** attributes set the size of the applet window.
- ✓ The **<param>** tag is used to pass parameters from HTML to the applet.
- ✓ The tag is now **deprecated** in modern browsers (replaced by Java Web Start and other technologies).

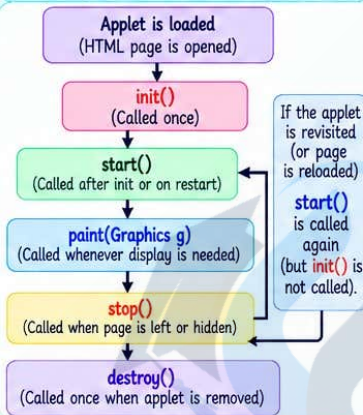
1. What are Applet Methods ?

- Applet methods are predefined methods in the `java.applet.Applet` class.
- These methods are **automatically called** by the browser or applet viewer at **different stages** of the applet's life cycle.
- The programmer **does not call** these methods explicitly.
- They are used to perform **initialization, execution, drawing, stopping** and **cleanup** of the applet.

2. List of Applet Methods

Method	Called When	Purpose
<code>init()</code>	Called once when the applet is first loaded.	Initialize variables, create objects, allocate resources.
<code>start()</code>	Called after <code>init()</code> or when the applet is restarted.	Start execution of the applet (resume).
<code>paint(Graphics g)</code>	Called whenever the applet needs to be displayed.	Draw output on the applet window using Graphics object.
<code>stop()</code>	Called when the applet becomes invisible or user leaves the page.	Stop execution and release resources (if needed).
<code>destroy()</code>	Called once when the applet is removed from memory.	Perform cleanup operations and free resources.

3. Life Cycle and Methods Execution Order



4. Description of Each Method

<code>init()</code>	- Called only once when the applet is loaded for the first time. - Used to initialize variables, create objects and set up resources. Example: Setting background color, loading images, etc.
<code>start()</code>	- Called after <code>init()</code> or when the applet is restarted. - Used to start or resume the execution. Example: Starting a thread, animation, etc.
<code>paint(Graphics g)</code>	- Called whenever the applet needs to be redrawn. - Used to display text, shapes, images, etc. - The Graphics object (g) is used for drawing.
<code>stop()</code>	- Called when the applet becomes invisible (e.g., user leaves the page). - Used to stop execution and release resources. Example: Stopping a thread or animation.
<code>destroy()</code>	- Called only once when the applet is removed from memory. - Used to perform final cleanup operations. Example: Releasing memory, closing files, etc.

5. Example Program Demonstrating All Applet Methods

```

import java.applet.Applet;
import java.awt.*;

public class AppletMethodsDemo extends Applet {
    public void init() {
        setBackground(Color.yellow);
        System.out.println("init() called - Applet initialized");
    }
    public void start() {
        System.out.println("start() called - Applet started");
    }
    public void paint(Graphics g) {
        g.setColor(Color.blue);
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.drawString("Welcome to Applet!", 50, 50);
        System.out.println("paint() called - Applet painted");
    }
    public void stop() {
        System.out.println("stop() called - Applet stopped");
    }
    public void destroy() {
        System.out.println("destroy() called - Applet destroyed");
    }
}
  
```

6. Output (Console and Applet Window)

Console Output
(in Applet Viewer)

```

init() called - Applet initialized
start() called - Applet started
paint() called - Applet painted
stop() called - Applet stopped
destroy() called - Applet destroyed
  
```

Applet Window Output

Welcome to Applet!

7. Key Points

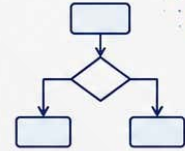
- ✓ Applet methods are predefined in `java.applet.Applet` class.
- ✓ These methods are automatically called by the browser or applet viewer.
- ✓ The programmer **does not** call them explicitly.
- ✓ The methods are called in a specific order: `init()` → `start()` → `paint()` → `stop()` → `destroy()`.
- ✓ `paint()` can be called multiple times.
- ✓ `start()` may be called again if the applet is revisited, but `init()` is not called again.
- ✓ These methods are used to manage the life cycle and behavior of the applet.



8. Interview Questions

- What are applet methods? Give their purpose.
- Explain the life cycle of an applet with diagram.
- Write a program to demonstrate all applet methods.
- Which method is called first? Why?
- Can `init()` be called more than once? Explain.
- How many times can `paint()` be called?
- What is the difference between `stop()` and `destroy()`?
- Why is `paint(Graphics g)` called multiple times?
- What happens when an applet is revisited?
- Are these methods defined in `java.applet.Applet` class or any other class?

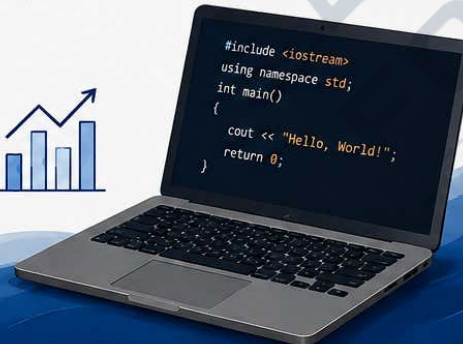
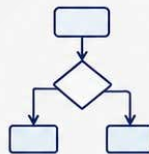
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



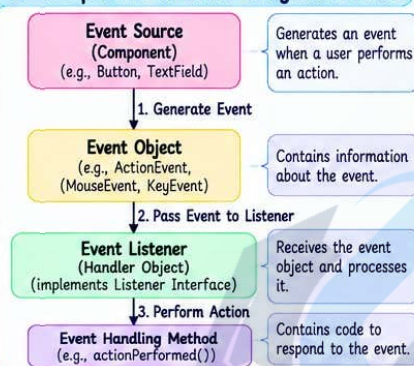
1. What is Event Handling ?

- Event handling is the mechanism to **handle user actions (events)** like button click, mouse movement, key press, etc. in a Java program.
- In applet programming, event handling allows the applet to **respond to user interactions**.
- Events are generated by **user components (source)** and are handled by **event listener objects**.

2. What is Event Delegation Model ?

- In Java, event handling is based on the **Event Delegation Model (EDM)**.
- The **source object** (e.g., Button) generates an event when a user performs an action.
- The **event object** is passed to one or more **listener (handler) objects**.
- The listener object processes the event and performs the appropriate action.
- This model is implemented using **interfaces** and follows the **"Delegation"** approach, where the source delegates the event to the listener.

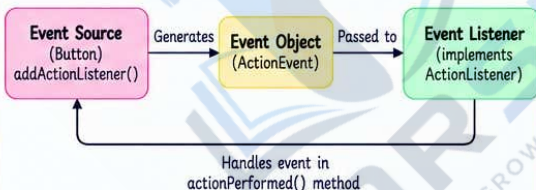
3. Components of Event Delegation Model



4. Working of Event Delegation Model

- 1 User performs an action (e.g., click) on a component (e.g., Button).
- 2 The component (event source) generates an event object (e.g., ActionEvent).
- 3 The event object is passed to the registered listener object.
- 4 The listener object's event handling method is called (e.g., actionPerformed()).
- 5 The method processes the event and performs the desired action.

5. Event Delegation Model Diagram



6. Common Event Listener Interfaces (for Applet)

Listener Interface	Event Type	Method (Event Handling Method)
ActionListener	Button click, Menu item	actionPerformed(ActionEvent e)
ItemListener	Checkbox, Choice, List	itemStateChanged(ItemEvent e)
MouseListener	Mouse click, press, release	mouseClicked(MouseEvent e) mousePressed(...), mouseReleased(...)
MouseMotionListener	Mouse move, drag	mouseMoved(MouseEvent e) mouseDragged(...)
KeyListener	Key press, release, type	keyPressed(KeyEvent e) keyReleased(...), keyTyped(...)
FocusListener	Focus gained or lost	focusGained(FocusEvent e) focusLost(...)

7. Example: Button Click Event in Applet

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class EventDemo extends Applet
implements ActionListener {
    Button b;
    public void init() {
        b = new Button("Click Me");
        b.addActionListener(this); // Register listener
    }
    public void actionPerformed(ActionEvent e) {
        showStatus("Button is clicked!");
    }
}
  
```

8. Explanation of the Example

- The Button is the **event source**.
- When the user clicks the button, an **ActionEvent** is generated.
- The event object is passed to the **actionPerformed()** method of the **listener** (this applet).
- The **showStatus()** method displays the message in the applet window's status bar.

9. Other Example (ItemEvent)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class ItemEventDemo extends Applet
implements ItemListener {
    Checkbox cb;
    public void init() {
        cb = new Checkbox("Java");
        cb.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent e) {
        showStatus("Checkbox state changed");
    }
}
  
```

10. Key Points

- ✓ Event handling in Java follows the **Event Delegation Model**.
- ✓ **Source object** generates an event.
- ✓ **Event object** contains event information.
- ✓ **Listener object** handles the event.
- ✓ Event handling is implemented using **listener interfaces**.
- ✓ A component can have multiple listeners.
- ✓ We must register the listener using **addActionListener()**, **addItemListener()**, etc.
- ✓ Each event has a specific **event class** (e.g., **ActionEvent**, **MouseEvent**).

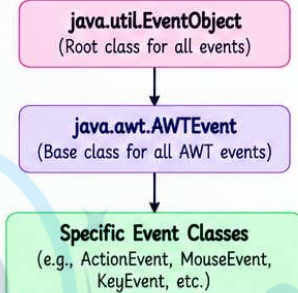
11. Advantages of Event Delegation Model

- ✓ Easy to handle user interaction.
- ✓ Flexible and extendable.
- ✓ Supports multiple listeners.
- ✓ Provides a standard mechanism using **interfaces**.
- ✓ Separation of event generation and handling increases modularity.
- ✓ Widely used in GUI programming (AWT/Swing).

1. What are Event Classes ?

- Event classes are predefined classes in the **java.awt.event** package that represent **different types of user events**.
- These classes carry information about the event (source, type, data, etc.) to the event listener.
- Each event class is a **subclass of the AWTEvent class**.
- They are used in event handling to **identify and process user actions** like mouse click, key press, window events, etc.

2. Hierarchy of Event Classes



3. Commonly Used Event Classes

Event Class	Package	Description
ActionEvent	java.awt.event	Generated when a button is clicked, menu item is selected, or any action is performed.
ItemEvent	java.awt.event	Generated when the state of a checkbox, choice list, or item is changed.
MouseEvent	java.awt.event	Generated for mouse actions like click, press, release, move, drag, etc.
KeyEvent	java.awt.event	Generated for keyboard actions like key press, release, and type.
WindowEvent	java.awt.event	Generated for window events like open, close, activate, deactivate, etc.
FocusEvent	java.awt.event	Generated when a component gains or loses the keyboard focus.
TextEvent	java.awt.event	Generated when text in a TextField or TextArea is changed.

4. Description of Important Event Classes

Event Class	Details
ActionEvent	- Occurs when a button is pressed or an action is performed. - Handled by ActionListener interface.
ItemEvent	- Occurs when a checkbox, choice or list item is selected or deselected. - Handled by ItemListener interface.
MouseEvent	- Occurs when mouse is clicked, pressed, released, moved or dragged. - Handled by MouseListener and MouseMotionListener.
KeyEvent	- Occurs when a key is pressed, released or typed. - Handled by KeyListener interface.
WindowEvent	- Occurs when a window is opened, closed, activated, deactivated, etc. - Handled by WindowListener interface.
FocusEvent	- Occurs when a component gains or loses focus. - Handled by FocusListener interface.
TextEvent	- Occurs when text in a text component is changed. - Handled by TextListener interface.

5. Example Programs for Different Event Classes

ActionEvent (Button Click)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class ActionDemo
extends Applet
implements ActionListener {
    Button b;
    public void init() {
        b = new Button("Click Me");
        b.addActionListener(this);
        add(b);
    }
    public void actionPerformed
    (ActionEvent e) {
        showStatus("Button clicked");
    }
}
  
```

MouseEvent (Mouse Click)

```

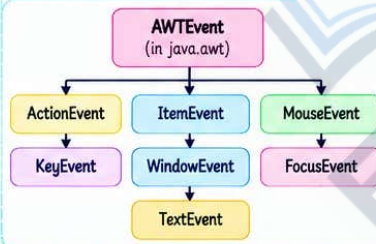
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class MouseDemo
extends Applet
implements MouseListener {
    public void init() {
        addMouseListener(this);
        setBackground(Color.yellow);
    }
    public void mouseClicked
    (MouseEvent e) {
        showStatus("Mouse Clicked");
    }
    public void mousePressed(MouseEvent e) {
        showStatus("Mouse Pressed");
    }
    public void mouseReleased(MouseEvent e) {
        showStatus("Mouse Released");
    }
    public void mouseEntered(MouseEvent e) {
        showStatus("Mouse Entered");
    }
    public void mouseExited(MouseEvent e) {
        showStatus("Mouse Exited");
    }
}
  
```

KeyEvent (Key Press)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class KeusDemo
extends Applet
implements KeyListener {
    public void init() {
        addKeyListener(this);
        setBackground(Color.lightGray);
    }
    public void keyPressed(KeyEvent e) {
        showStatus("Key Pressed: " +
            e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {
        showStatus("Key Released: " +
            e.getKeyChar());
    }
    public void keyTyped(KeyEvent e) {
        showStatus("Key Typed: " +
            e.getKeyChar());
    }
}
  
```

6. Event Class Relationships



7. Key Points

- ✓ Event classes are present in **java.awt.event** package.
- ✓ All event classes are subclasses of **AWTEvent**.
- ✓ Each event class carries information about the event.
- ✓ Different events are handled by different listener interfaces.
- ✓ Event classes are used to implement the event delegation model.
- ✓ Common event classes: **ActionEvent, ItemEvent, MouseEvent, KeyEvent, WindowEvent, FocusEvent, TextEvent**.
- ✓ These classes help in creating interactive applet applications.

8. Uses / Applications

- Used to handle user interactions in applets.
- Helps in creating interactive GUI applications.
- Used in games, animations and educational applets.
- Makes applets respond to mouse, keyboard and other events.

9. Interview Questions

- What are event classes in Java?
- Name some commonly used event classes.
- Which package contains event classes?
- What is the superclass of all event classes?
- Explain the difference between ActionEvent and MouseEvent.
- Which listener interface is used for handling KeyEvent?

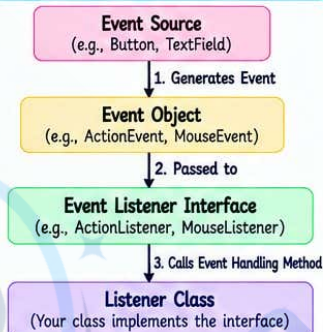
10. Summary

Event classes in Java (**java.awt.event** package) represent different types of user events. They are subclasses of the **AWTEvent** class and are used in the **event delegation model**. Each event class provides specific information about the event and helps in handling user interactions in applet applications.

1. What are Event Listener Interfaces ?

- Event Listener Interfaces are interfaces provided in the **java.awt.event package**.
- These interfaces contain one or more **event handling methods**.
- A class implements these interfaces to **receive and process specific events** generated by components like Button, TextField, etc.
- When an event occurs, the event source object **notifies all registered listener objects** by calling the appropriate method.

2. How Event Listener Interfaces Work ?



3. Syntax to Implement a Listener Interface

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class Example extends Applet
implements ActionListener {
    Button b;
    public void init() {
        b = new Button("Click Me");
        b.addActionListener(this);
        add(b);
    }
    public void actionPerformed(ActionEvent e) {
        // code to handle event
    }
}
  
```

4. Common Event Listener Interfaces

Listener Interface	Event Type Handled	Main Method(s)
ActionListener	Button click, Menu item selection	actionPerformed (ActionEvent e)
ItemListener	Checkbox, Choice, List selection	itemStateChanged (ItemEvent e)
MouseListener	Mouse click, press, release, enter, exit	mouseClicked(), mousePressed(), mouseReleased(), mouseEntered(), mouseExited()
MouseMotionListener	Mouse move, drag	mouseMoved(), mouseDragged()
KeyListener	Key press, release, type	keyPressed(), keyReleased(), keyTyped()
WindowListener	Window open, close, activate, deactivate	windowOpened(), windowClosing(), etc.
FocusListener	Focus gain or lost	focusGained(), focusLost()
TextListener	Text change in TextField or TextArea	textValueChanged (TextEvent e)

5. ActionListener Interface

- Used to handle **action events** like button click.
- Contains a single method: **actionPerformed(ActionEvent e)**

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class ActionDemo extends Applet
implements ActionListener {
    Button b;
    public void init() {
        b = new Button("Click Me");
        b.addActionListener(this);
        add(b);
    }
    public void actionPerformed(ActionEvent e) {
        showStatus("Button Clicked!");
    }
}
  
```

6. MouseListener Interface

- Used to handle **mouse events** like click, press, release, enter and exit.
- Contains the following methods:

```

▶ mouseClicked(MouseEvent e)
▶ mousePressed(MouseEvent e)
▶ mouseReleased(MouseEvent e)
▶ mouseEntered(MouseEvent e)
▶ mouseExited(MouseEvent e)

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class MouseDemo extends Applet
implements MouseListener {
    public void init() {
        addMouseListener(this);
    }
    public void mouseClicked(MouseEvent e) {
        showStatus("Mouse Clicked");
    }
    public void mousePressed(MouseEvent e) {
        showStatus("Mouse Pressed");
    }
    public void mouseReleased(MouseEvent e) {
        showStatus("Mouse Released");
    }
    public void mouseEntered(MouseEvent e) {
        showStatus("Mouse Entered");
    }
    public void mouseExited(MouseEvent e) {
        showStatus("Mouse Exited");
    }
}
  
```

7. KeyListener Interface

- Used to handle **keyboard events**.
- Contains the following methods:

```

▶ keyPressed(KeyEvent e)
▶ keyReleased(KeyEvent e)
▶ keyTyped(KeyEvent e)

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class KeyDemo extends Applet
implements KeyListener {
    public void init() {
        addKeyListener(this);
    }
    public void keyPressed(KeyEvent e) {
        showStatus("Key Pressed: " + e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {
        showStatus("Key Released: " + e.getKeyChar());
    }
    public void keyTyped(KeyEvent e) {
        showStatus("Key Typed: " + e.getKeyChar());
    }
}
  
```

8. WindowListener Interface

- Used to handle window events in applet or frame.
- Contains methods like:

```

▶ windowOpened(WindowEvent e)
▶ windowClosing(WindowEvent e)
▶ windowClosed(WindowEvent e)
▶ windowIconified(WindowEvent e)
▶ windowDeiconified(WindowEvent e)
▶ windowActivated(WindowEvent e)
▶ windowDeactivated(WindowEvent e)

public void windowClosing(WindowEvent e){
    showStatus("Window is closing");
}
  
```

9. FocusListener Interface

- Used to handle **focus events**.
- Contains methods:

```

▶ focusGained(FocusEvent e)
▶ focusLost(FocusEvent e)
  
```

10. TextListener Interface

- Used to handle **text change events** in TextField or TextArea.
- Contains method:

```

▶ textValueChanged(TextEvent e)
  
```

11. Key Points

- Event listener interfaces are defined in **java.awt.event package**.
- They are used to handle specific user events.
- A class implements the required listener interface.
- Each interface has one or more event handling methods.
- The event source notifies the registered listener object.
- Common interfaces: **ActionListener, MouseListener, WindowListener, FocusListener, TextListener**.

12. Uses / Applications

- Handling user interactions in applets.
- Creating interactive GUI applications.
- Used in games, educational tools, etc.
- Helps in making applications responsive to user actions.



1. Introduction

- Mouse and keyboard events are user actions generated by input devices.
- These events are handled in Java applet programming using event listener interfaces provided in **java.awt.event package**.
- For mouse events → we use **MouseListener** and **MouseMotionListener** interfaces.
- For keyboard events → we use **KeyListener** interface.

2. Mouse Events and Listener Interfaces

- Mouse events are generated when the user moves, clicks, presses or releases the mouse.
- In Java, mouse events are handled using two interfaces:

MouseListener → Handles mouse click, press, release, enter, exit events.

MouseMotionListener → Handles mouse move and mouse drag events.

3. Mouse Event Methods

Method	Event Generated	Description
mouseClicked(MouseEvent e)	Mouse click	Invoked when the mouse is clicked.
mousePressed(MouseEvent e)	Mouse press	Invoked when a mouse button is pressed.
mouseReleased(MouseEvent e)	Mouse release	Invoked when the mouse button is released.
mouseEntered(MouseEvent e)	Mouse enters component	Invoked when the mouse pointer enters the component.
mouseExited(MouseEvent e)	Mouse exits component	Invoked when the mouse pointer leaves the component.
mouseMoved(MouseEvent e)	Mouse move	Invoked when the mouse is moved (without dragging).
mouseDragged(MouseEvent e)	Mouse drag	Invoked when the mouse is dragged (with button pressed).

4. Example: Handling Mouse Events in Applet

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class MouseEventDemo extends Applet
implements MouseListener, MouseMotionListener {
    String msg = "Move the Mouse";
    int x, y;
    public void init() {
        addMouseListener(this);
        addMouseMotionListener(this);
        setBackground(Color.white);
    }
    public void paint(Graphics g) {
        g.setColor(Color.blue);
        g.drawString(msg + " at (" + x + ", " + y + ")", 20, 50);
    }
    // MouseListener methods
    public void mouseClicked(MouseEvent e) { msg = "Mouse Clicked"; }
    public void mousePressed(MouseEvent e) { msg = "Mouse Pressed"; }
    public void mouseReleased(MouseEvent e) { msg = "Mouse Released"; }
    public void mouseEntered(MouseEvent e) { msg = "Mouse Entered"; }
    public void mouseExited(MouseEvent e) { msg = "Mouse Exited"; }
    // MouseMotionListener methods
    public void mouseMoved(MouseEvent e) { x = e.getX(); y = e.getY(); msg = "Mouse Moved"; repaint(); }
    public void mouseDragged(MouseEvent e) { x = e.getX(); y = e.getY(); msg = "Mouse Dragged"; repaint(); }
}
```

5. Output (Mouse Events)



6. Keyboard Events and Listener Interface

- Keyboard events are generated when the user presses or releases a key.
- In Java, keyboard events are handled using **KeyListener** interface.
- It provides three methods to handle key press, key release and key type events.

7. KeyListener Interface Methods

Method	Event Generated	Description
KeyPressed(KeyEvent e)	Key press	Invoked when a key is pressed.
keyReleased(KeyEvent e)	Key release	Invoked when a key is released.
KeyTyped(KeyEvent e)	Key type	Invoked when a key is typed (character generated).

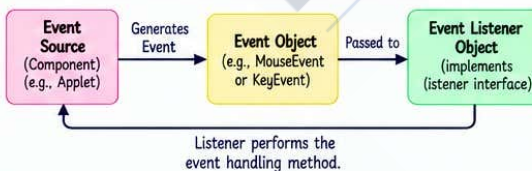
8. Example: Handling Keyboard Events in Applet

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class KeyEventDemo extends Applet
implements KeyListener {
    String msg = "Press any key";
    public void init() {
        addKeyListener(this);
        setBackground(Color.white);
    }
    public void paint(Graphics g) {
        g.setColor(Color.red);
        g.drawString(msg, 20, 50);
    }
    // KeyListener methods
    public void keyPressed(KeyEvent e) {
        msg = "Key Pressed : " + e.getKeyChar();
        repaint();
    }
    public void keyReleased(KeyEvent e) {
        msg = "Key Released : " + e.getKeyChar();
        repaint();
    }
    public void keyTyped(KeyEvent e) {
        msg = "Key Typed : " + e.getKeyChar();
        repaint();
    }
}
```

9. Output (Keyboard Events)



10. Event Handling Process for Mouse and Keyboard



11. Key Points

- ✓ Mouse events are handled using **MouseListener** and **MouseMotionListener** interfaces.
- ✓ Keyboard events are handled using **KeyListener** interface.
- ✓ Each listener interface provides specific methods to handle different events.
- ✓ The event object (**MouseEvent / KeyEvent**) contains information about the event.
- ✓ To receive events, the applet must register the listener using **addMouseListener()**, **addMouseMotionListener()** or **addKeyListener()**.
- ✓ These events make applets interactive.

1. What are Adapter Classes ?

- Adapter classes are special classes provided in the **java.awt.event package**.
- These classes provide **empty** (default) **implementations** of all methods of a listener interface.
- They are used to **overcome the problem** of implementing all methods of a listener interface.
- By extending an adapter class, we **only need to override** the methods that we are interested in.
- Adapter classes are useful for **event handling in applet programming**.

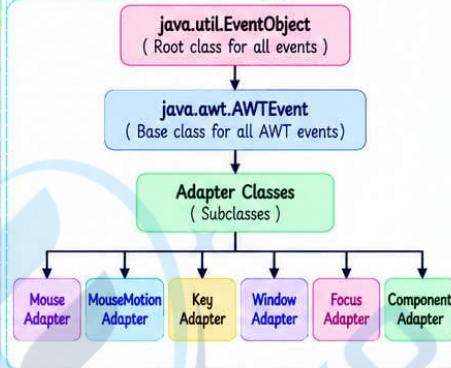
2. Why Adapter Classes are Used ?

- Listener interfaces contain **multiple methods**.
- If we implement a listener interface directly, we must provide implementation for **all methods**, even if we need only one or two of them.
- Adapter classes provide default (**empty**) implementation for all methods, so we can override **only the required methods**.
- This makes the code **simple, readable and easy to maintain**.

3. List of Common Adapter Classes

Adapter Class	Listener Interface	Purpose
MouseAdapter	MouseListener	Provides empty implementation of all MouseListener methods.
MouseMotionAdapter	MouseMotionListener	Provides empty implementation of all MouseMotionListener methods.
KeyAdapter	KeyListener	Provides empty implementation of all KeyListener methods.
WindowAdapter	WindowListener	Provides empty implementation of all WindowListener methods.
FocusAdapter	FocusListener	Provides empty implementation of all FocusListener methods.
ComponentAdapter	ComponentListener	Provides empty implementation of all ComponentListener methods.

4. Adapter Class Hierarchy

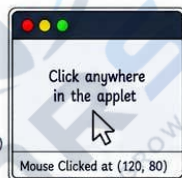


5. MouseAdapter Example (Handling Mouse Click)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class MouseAdapterDemo extends Applet {
    public void init() {
        addMouseListener(new MyMouseAdapter());
    }
    class MyMouseAdapter extends MouseAdapter {
        public void mouseClicked(MouseEvent e) {
            showStatus("Mouse Clicked at (" + e.getX() + ", " + e.getY() + ")");
        }
    }
}
  
```



6. KeyAdapter Example (Handling Key Events)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class KeyAdapterDemo extends Applet {
    public void init() {
        addKeyListener(new MyKeyAdapter());
    }
    class MyKeyAdapter extends KeyAdapter {
        public void keyPressed(KeyEvent e) {
            showStatus("Key Pressed : " + e.getKeyChar());
        }
    }
}
  
```

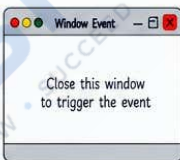


7. WindowAdapter Example (Handling Window Events)

```

import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

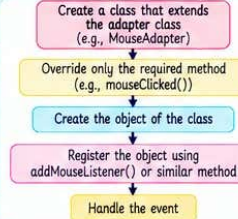
public class WindowAdapterDemo extends Applet {
    public void init() {
        Frame f = new Frame("Window Event");
        f.setSize(300, 200);
        f.addWindowListener(new MyWindowAdapter());
        f.setVisible(true);
    }
    class MyWindowAdapter extends WindowAdapter {
        public void windowClosing(WindowEvent e) {
            showStatus("Window is closing...");
            System.exit(0);
        }
    }
}
  
```



8. Comparison: Listener Interface vs Adapter Class

Feature	Listener Interface	Adapter Class
Methods	All methods must be implemented	Provides empty implementation
Implementation	Compulsory for all methods	Override only required methods
Code Size	More code	Less code
Usage	When we need to handle all methods	When we need to handle only specific methods
Example	MouseListener, KeyListener	MouseAdapter, KeyAdapter

9. Flow of Using Adapter Class



10. Advantages of Adapter Classes

- ✓ Reduces the need to implement all methods.
- ✓ Simplifies event handling code.
- ✓ More readable and maintainable.
- ✓ Useful when only specific events are required.
- ✓ Widely used in applet and GUI programming.

11. Key Points

- ✓ Adapter classes are defined in **java.awt.event package**.
- ✓ They provide empty implementation of listener interface methods.
- ✓ We only override the required methods.
- ✓ Common adapter classes: **MouseAdapter, MouseMotionAdapter, KeyAdapter, WindowAdapter, FocusAdapter, ComponentAdapter**.
- ✓ They help in simple and efficient event handling in applets.

12. Interview Questions

- 1 What are adapter classes? Why are they used?
- 2 Name the commonly used adapter classes.
- 3 Explain MouseAdapter with example.
- 4 Explain KeyAdapter with example.
- 5 Differentiate between listener interfaces and adapter classes.

1. What is AWT ?

- AWT (Abstract Window Toolkit) is a GUI (Graphical User Interface) toolkit provided by Java.
- It is part of the **java.awt package**.
- It provides a set of classes to create window-based applications (GUI).
- AWT is **platform-independent** (write once, run anywhere).
- It is **heavyweight** because it uses the native operating system resources for creating its components.

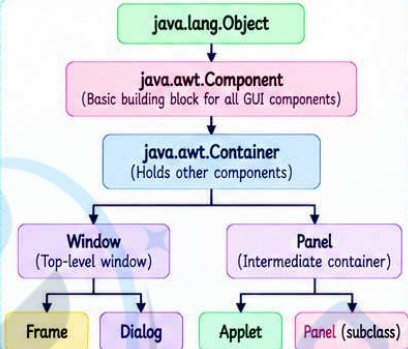
2. Features of AWT

- Provides a rich set of GUI components like Button, Label, TextField, etc.
- Platform-independent (same code runs on any OS).
- Uses native operating system resources (heavyweight components).
- Supports event handling.
- Simple and easy to use.
- Provides layout managers for arranging components.
- Supports applets and applications.
- Can be used to create window-based interfaces.

3. AWT vs Swing

	AWT	Swing
Part of	java.awt package	javax.swing package
Component type	Heavyweight (native OS components)	Lightweight (pure Java components)
Platform dependence	Uses OS resources	Platform-independent (does not use native OS)
Look and Feel	Depends on OS	Provides customizable look and feel
Component set	Limited components	More advanced components
Performance	Slower	Faster
Example components	Button, Label, TextField, Frame, Panel	JButton, JLabel, JTextField, JFrame, JPanel

4. AWT Class Hierarchy



5. Important AWT Packages

Package	Description
java.awt	Contains all core AWT classes like Component, Container, Button, Label, etc.
java.awt.event	Contains event classes and listener interfaces for event handling.
java.applet	Used for creating applets. Applet is a subclass of Panel and belongs to AWT.

6. Commonly Used AWT Components

Component	Description
Label	Displays read-only text.
Button	A clickable button.
TextField	A single-line text input field.
TextArea	A multi-line text input area.
Checkbox	A selectable option (on/off).
CheckboxGroup	Group of checkboxes (only one can be selected).
Choice	Dropdown list of options.
List	A list of items (multiple selection possible).
Panel	A container to hold other components.
Frame	A top-level window with title bar, menu, etc.

7. A Simple AWT Applet Example

```

import java.applet.Applet;
import java.awt.*;
public class AwtDemo extends Applet {
    public void init() {
        setLayout(new FlowLayout());
        Label l = new Label("Welcome to AWT");
        Button b = new Button("Click Me");
        add(l);
        add(b);
    }
}
  
```

Output (Applet in Browser)



8. Working of AWT

- AWT provides classes to create and manage GUI components.
- Components are added to a container (like Frame or Panel).
- Layout managers are used to arrange components.
- Event handling is used to handle user interactions (click, key press, etc.).
- In an applet, the **init()** method is used to create and add AWT components.

9. AWT in Applet vs Application

AWT in Applet	AWT in Application
• Applet is a subclass of Panel from AWT. • The init() method is used to create and add components. • Runs inside a browser or applet viewer. • Example: Using Button, Label in applet.	• Uses a Frame as the top-level window. • The main() method is used to create components. • Runs as a standalone program. • Example: Creating a window with Frame and adding components.

10. Advantages and Disadvantages

Advantages	Disadvantages
✓ Simple and easy to use. ✓ Platform-independent. ✓ Uses native OS components (better integration). ✓ Provides essential GUI components. ✓ Supports event handling and layout managers.	✗ Limited set of components compared to Swing. ✗ Heavyweight (performance may be slower). ✗ Look and feel depends on operating system. ✗ Less flexible and less customizable.

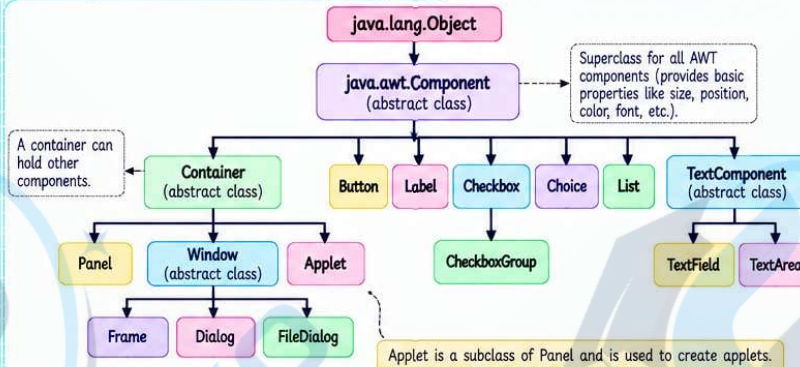
11. Key Points

- ✓ AWT (Abstract Window Toolkit) is a GUI toolkit in Java.
- ✓ It is part of the **java.awt package**.
- ✓ Provides classes for creating GUI components.
- ✓ Uses native OS resources (heavyweight).
- ✓ Supports event handling.
- ✓ Used for both applets and applications.

1. What is AWT Class Hierarchy ?

- AWT (Abstract Window Toolkit) provides a set of classes for creating graphical user interfaces in Java.
- The classes in AWT are arranged in a hierarchical form, which is called **AWT class hierarchy**.
- The root of the hierarchy is the **java.lang.Object** class.
- From this class, many classes are derived to create components, containers and windows.
- Applet class is also part of this hierarchy and is used to create applets.

2. Complete AWT Class Hierarchy (Diagram)



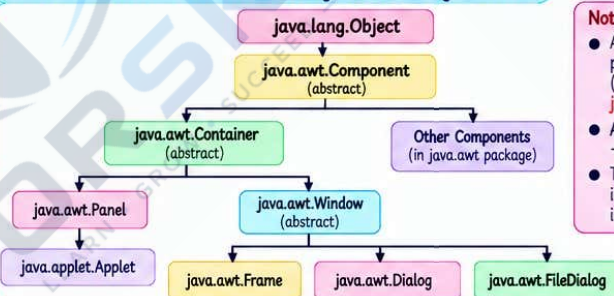
3. Explanation of Main Classes

Class	Description
Object	Root class of all Java classes.
Component	Abstract class. Root of all AWT components. Provides basic GUI properties and methods.
Container	Abstract class. Can hold other components.
Panel	A container that can hold components. Default layout: FlowLayout.
Applet	Subclass of Panel. Used to create applets. Runs inside a browser or applet viewer.
Window	Abstract class for top-level windows.
Frame	A top-level window with title bar, menu, etc.
Dialog	A secondary window.
FileDialog	A dialog for file selection.
Button	A clickable button.
Label	Displays read-only text.
Checkbox	A selectable option (on/off).
CheckboxGroup	Group of checkboxes (only one can be selected).
Choice	A drop-down list.
List	A list of items (multiple selection).
TextComponent	Abstract class for text components.
TextField	A single-line text input field.
TextArea	A multi-line text input field.

4. Important Points

- ✓ The root class of AWT hierarchy is **java.lang.Object**.
- ✓ Component is the base class for all visual components.
- ✓ Container is a subclass of Component.
- ✓ Panel and **Applet** are subclasses of Container.
- ✓ Applet is a subclass of Panel.
- ✓ Window is a subclass of Container and has **Frame**, **Dialog** and **FileDialog**.
- ✓ TextField and **TextArea** are subclasses of **TextComponent**.
- ✓ All AWT components inherit methods from Component class.

5. Partial View of Class Hierarchy with Package Names



Note:

- All these classes are present in **java.awt** package (except Applet, which is in **java.applet** package).
- **Object** → **Panel** → **Container** → **Component** → **Object**
- This hierarchy shows inheritance (IS-A relationship) in Java.

6. Package Information

Package	Contains
java.awt	Contains all main AWT classes (Component, Container, Button, Label, etc.).
java.applet	Contains the Applet class.

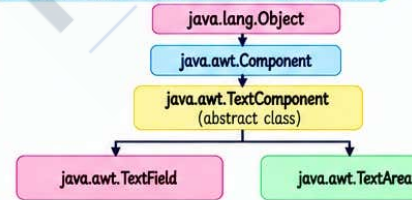
7. Example: Using Some AWT Classes in Hierarchy

```

import java.applet.Applet;
import java.awt.*;
public class DemoHierarchy extends Applet {
    public void init() {
        Button b = new Button("Click"); // Button (extends Component)
        Label l = new Label("AWT Example"); // Label (extends Component)
        add(b); // add to Applet (which is a Panel → Container)
        add(l);
    }
}

```

8. Sample Hierarchy for Text Components



1. Components in AWT

- A Component is the **basic building block** of any AWT graphical user interface.
- It represents a **visual object** that can be displayed on the screen.
- All GUI controls like Button, Label, TextField, Checkbox etc. are components.
- Component class is in the **java.awt package** and is the superclass of all AWT components.

```
public class Component
    extends Object
```

2. Common AWT Components

Component	Description	Example
Button	A clickable button.	<code>new Button("Click")</code>
Label	Displays read-only text.	<code>new Label("Name")</code>
TextField	Single-line text input.	<code>new TextField(20)</code>
TextArea	Multi-line text input.	<code>new TextArea(5, 20)</code>
Checkbox	A selectable option (on/off).	<code>new Checkbox("Male")</code>
CheckboxGroup	Group of checkboxes (radio buttons).	<code>new CheckboxGroup()</code>
Choice	A drop-down list.	<code>new Choice()</code>
List	A list of items.	<code>new List(4)</code>
Scrollbar	A scroll bar control.	<code>new Scrollbar()</code>
Canvas	Provides a blank area for custom drawing.	<code>extends Canvas</code>
Panel	A container to hold components.	<code>new Panel()</code>

3. Containers in AWT

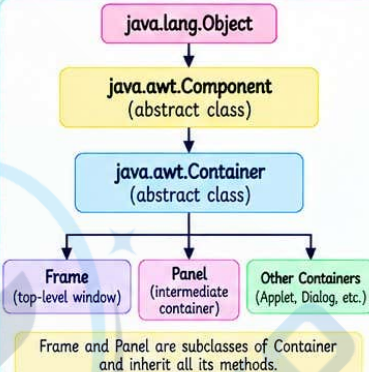
- A Container is a special type of component that **can hold other components**.
- Container class is an **abstract class**.
- Provides methods to add, remove and arrange components.
- Container is also in **java.awt package**.

```
public class Container
    extends Component
```

Important Methods of Container

Method	Description
<code>add(Component c)</code>	Adds a component to the container.
<code>remove(Component c)</code>	Removes a component from the container.
<code>setLayout(LayoutManager lm)</code>	Sets layout manager for arranging components.
<code>getComponents()</code>	Returns all components in the container.

4. AWT Class Hierarchy (Relevant Part)



5. Frames in AWT

- Frame is a top-level window with title bar, menu bar, border and close, minimize, maximize buttons.
- It is a subclass of **java.awt.Window** which is a subclass of **Container**.
- Used to create stand-alone window applications (also used in applet display, but mainly in applications).

```
public class Frame
    extends Window
```

6. Important Methods of Frame

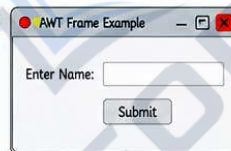
Method	Description
<code>Frame(String title)</code>	Creates a frame with title.
<code>setSize(int w, int h)</code>	Sets width and height.
<code>setTitle(String title)</code>	Sets the title of the frame.
<code>setVisible(boolean b)</code>	Shows or hides the frame.
<code>setLayout(LayoutManager lm)</code>	Sets layout manager.
<code>add(Component c)</code>	Adds a component to frame.
<code>setBackground(Color c)</code>	Sets background color.
<code>setResizable(boolean b)</code>	Enables or disables resizing.
<code>setLocation(int x, int y)</code>	Sets the position of frame.
<code>dispose()</code>	Closes and releases resources.

7. Example: Frame with Components

```
import java.awt.*;
public class FrameDemo {
    public static void main(String[] args) {
        Frame f = new Frame("AWT Frame Example");
        f.setSize(400, 250);
        f.setLayout(new FlowLayout());

        Label l1 = new Label("Enter Name:");
        TextField tf = new TextField(15);
        Button b = new Button("Submit");

        f.add(l1);
        f.add(tf);
        f.add(b);
        f.setVisible(true);
    }
}
```



8. Panels in AWT

- Panel is an intermediate container used to hold and arrange components.
- It does **not** have its **own** window (no title bar).
- It is a subclass of **Container**.
- Used inside a Frame, Applet or another Panel.
- Helps in organizing components in a group.

```
public class Panel
    extends Container
```

9. Important Methods of Panel

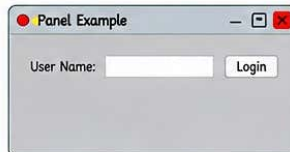
Method	Description
<code>Panel()</code>	Creates a panel.
<code>setLayout(LayoutManager lm)</code>	Sets layout manager for the panel.
<code>add(Component c)</code>	Adds a component to the panel.
<code>remove(Component c)</code>	Removes a component from the panel.
<code>setBackground(Color c)</code>	Sets background color.
<code>getComponents()</code>	Returns all components in the panel.

10. Example: Using Panel inside a Frame

```
import java.awt.*;
public class PanelDemo {
    public static void main(String[] args) {
        Frame f = new Frame("Panel Example");
        f.setSize(400, 300);
        f.setLayout(new BorderLayout());

        Panel p = new Panel();
        p.setBackground(Color.lightGray);
        p.setLayout(new FlowLayout());
        p.add(new Label("User Name:"));
        p.add(new TextField(15));
        p.add(new Button("Login"));

        f.add(p, BorderLayout.CENTER);
        f.setVisible(true);
    }
}
```



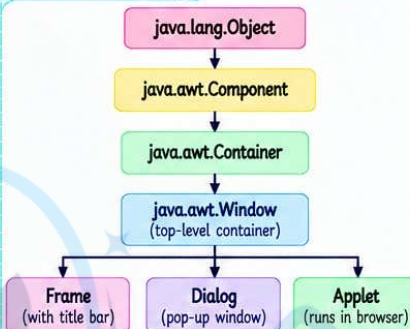
11. Key Points

- ✓ Component is the basic GUI element.
- ✓ Container can hold multiple components.
- ✓ Frame is a top-level window (subclass of Window).
- ✓ Panel is an intermediate container (subclass of Container).
- ✓ Both Frame and Panel inherit methods from Container.
- ✓ Use `add()` method to add components.
- ✓ Use `setLayout()` to arrange components.
- ✓ Common layout managers: FlowLayout, BorderLayout, GridLayout, etc.
- ✓ Panels are useful for grouping related components.

1. What is Window ?

- Window is a **top-level container** in the AWT (Abstract Window Toolkit).
- It represents a separate window on the screen with its own title bar, border and close buttons.
- Window is a direct subclass of **java.awt.Container**.
- Unlike Frame, a Window does **not have a title bar** and menu bar by default. It is usually used as a base class for creating other top-level windows like **Frame**, **Dialog**, etc.

2. Class Hierarchy



3. Important Constructors of Window

Constructor	Description
Window(Frame owner)	Creates a window with the specified owner frame.
Window(Window owner)	Creates a window with the specified owner window.
Window(Frame owner, GraphicsConfiguration gc)	Creates a window with the specified owner and graphics configuration.
Window(Window owner, GraphicsConfiguration gc)	Creates a window with the specified owner and graphics configuration.

4. Important Methods of Window

Method	Description
setSize(int width, int height)	Sets the width and height of the window.
setLocation(int x, int y)	Sets the position of the window on the screen.
setVisible(boolean b)	Shows or hides the window.
add(Component c)	Adds a component to the window.
remove(Component c)	Removes a component from the window.
dispose()	Closes and releases the native resources used by the window.
setBackground(Color c)	Sets the background color of the window.
setLayout(LayoutManager lm)	Sets the layout manager for arranging components.

5. Key Points about Window

- Window is a **top-level container** in AWT.
- It has **no title bar** and menu bar (unlike Frame).
- It is a subclass of **Container**.
- Used as the base class for Frame, Dialog and other top-level windows.
- Can contain other components like Button, Label, Panel, etc.
- Requires a parent window (owner) to be created.
- Provides multiple constructors for specifying the owner window.
- Useful when you need a custom top-level window without a title bar.

6. Example 1: Creating a Simple Window (Applet)

```

import java.applet.Applet;
import java.awt.*;
public class WindowExample extends Applet {
    public void init() {
        // Create a Window with null owner
        Window w = new Window((Frame)null);
        w.setSize(300, 200);
        w.setLayout(new FlowLayout());
        Label l = new Label("This is a Window");
        Button b = new Button("Close");
        w.add(l);
        w.add(b);
        w.setBackground(Color.lightGray);
        w.setVisible(true);
        // Close window on button click
        b.addActionListener(e -> w.dispose());
    }
}
  
```



Output:

- A window without title bar.
- Displays a label and a close button.
- Clicking the button closes the window.

7. Example 2: Using Owner Frame

```

import java.applet.Applet;
import java.awt.*;
public class WindowOwnerExample extends Applet {
    public void init() {
        Frame f = new Frame("Main Frame");
        f.setSize(300, 200);
        f.setLayout(new FlowLayout());
        Button btn = new Button("Open Window");
        f.add(btn);
        f.setVisible(true);
        btn.addActionListener(e -> {
            Window w = new Window(f); // owner frame
            w.setSize(250, 150);
            w.add(new Label("Child Window"));
            w.setVisible(true);
        });
    }
}
  
```



8. Window vs Frame (Difference)

Feature	Window	Frame
Title bar	No title bar	Has title bar
Menu bar	No menu bar	Has menu bar
Subclass of	java.awt.Container	java.awt.Window
Usage	Base class for top-level windows	Creates a window with title bar and menu bar
Components	Can hold other components	Can hold other components
Commonly used	Less commonly used directly	More commonly used

9. Common Use Cases

- Creating custom top-level windows without a title bar.
- Used as a base class for Frame and Dialog.
- Creating **splash screens**.
- Creating **pop-up windows** or tool windows.
- Used in combination with **owner window** for modal or modeless windows.

10. Important Notes

- Window** must have an **owner window** (Frame or another Window).
- It does not provide **title bar** and menu bar.
- It is useful for advanced GUI requirements where a simple **top-level window** is needed.
- Frame** and **Dialog** are built on top of **Window**.
- Always call **dispose()** to close the window and release resources.

11. Summary

- Window** is a top-level container in AWT.
- No title bar or menu bar.
- Subclass of **Container**.
- Used as a base class for **Frame** and **Dialog**.
- Can contain other components.
- Useful for creating custom top-level windows and pop-ups.
- Requires an **owner window**.

1. AWT Controls (Introduction)

- AWT provides a set of ready-to-use components called **controls** which are used to create graphical user interfaces (GUIs).
- Common controls include **Button, Label, TextField, Checkbox, Choice, List**, etc.
- These controls are part of the **java.awt package**.
- Controls are added to **container** (like Frame or Panel).
- They generate **events** (e.g., button click, text input) and are used with event handling.

2. Button Control

- Button** is used to perform an action when clicked.
- Class: **java.awt.Button**
- Common constructors:

```
Button() // creates a button with no label
Button(String label) // creates a button with label
```

Important methods:

Method	Description
setLabel(String)	Sets the label of the button.
getLabel()	Returns the label.
setEnabled(boolean)	Enables or disables the button.
addActionListener()	Registers action listener for button click event.

Example: Button

```
Button b = new Button("Click Me");
b.setBounds(50, 50, 100, 30);
add(b);
```



3. Label Control

- Label** is used to display a single line of read-only text.
- Class: **java.awt.Label**
- Common constructors:

```
Label() // empty label
Label(String text) // label with text
Label(String text, int alignment) // alignment: LEFT, CENTER, RIGHT
```

Important methods:

Method	Description
setText(String)	Sets the text of the label.
getText()	Returns the text.
setAlignment(int)	Sets text alignment (LEFT, CENTER, RIGHT).

Example: Label

```
Label l = new Label("Welcome to AWT");
l.setBounds(50, 50, 200, 30);
add(l);
```



4. Text Field Control

- TextField** is used to accept **single line text input** from the user.
- Class: **java.awt.TextField**
- Common constructors:

```
TextField() // empty text field
TextField(int columns) // specified width
TextField(String text) // with initial text
TextField(String text, int columns)
```

Important methods:

Method	Description
setText(String)	Sets the text in the text field.
getText()	Returns the text entered by user.
setEditable(boolean)	Makes it editable or read-only.
setEchoChar(char)	Displays echo character (e.g., *) for password field.

Example: Text Field

```
TextField tf = new TextField(20);
tf.setBounds(50, 50, 200, 30);
add(tf);
```



5. Common AWT Controls (List)

Control	Class	Purpose
Button	java.awt.Button	Performs an action when clicked.
Label	java.awt.Label	Displays read-only text.
TextField	java.awt.TextField	Accepts single line text input.
TextArea	java.awt.TextArea	Accepts multi-line text input.
Checkbox	java.awt.Checkbox	A selectable option (on/off).
Choice	java.awt.Choice	A drop-down list.
List	java.awt.List	A list of items (multiple selection possible).
Scrollbar	java.awt.Scrollbar	A scroll bar control.
Canvas	java.awt.Canvas	A blank area for custom drawing.

9. BorderLayout Example

```
setLayout(new BorderLayout());
add(new Button("North"), BorderLayout.NORTH);
add(new Button("South"), BorderLayout.SOUTH);
add(new Button("East"), BorderLayout.EAST);
add(new Button("West"), BorderLayout.WEST);
add(new Button("Center"), BorderLayout.CENTER);
```



6. Layout Managers (Introduction)

- Layout Manager** automatically arranges the components in a container.
- It controls the size and position of components.
- When a layout manager is used, we **do not need** to set the position and size manually using **setBounds()**.
- AWT provides several layout managers in **java.awt package**.
- Commonly used layout managers: **FlowLayout, BorderLayout, GridLayout, CardLayout**, etc.

10. GridLayout Example

```
setLayout(new GridLayout(3,2));
add(new Button("1"));
add(new Button("2"));
add(new Button("3"));
add(new Button("4"));
add(new Button("5"));
add(new Button("6"));
```



7. Types of Layout Managers

Layout Manager	Description
FlowLayout	Arranges components from left to right in a flow, automatically wrapping.
BorderLayout	Divides the container into five regions: North, South, East, West, and Center.
GridLayout	Arranges components in a rectangular grid of rows and columns.
CardLayout	Manages multiple components (like cards) but only one is visible at a time.
GridBagLayout	More flexible and advanced layout manager.

11. CardLayout Example

```
CardLayout card = new CardLayout();
setLayout(card);
add(new Button("First"), "one");
add(new Button("Second"), "two");
card.show(this, "one");
```



8. FlowLayout Example

- Places components in a left-to-right flow.
- Default layout for a Panel.

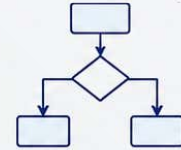
```
Panel p = new Panel();
p.setLayout(new FlowLayout());
p.add(new Button("OK"));
p.add(new Button("Cancel"));
p.add(new Button("Exit"));
add(p);
```



12. Key Points

- ✓ Button is used to perform an action.
- ✓ Label is used to display text.
- ✓ TextField is used for single line input.
- ✓ Layout manager automatically arranges components.
- ✓ Common layout managers: FlowLayout, BorderLayout, GridLayout, CardLayout.
- ✓ Using layout managers makes GUI responsive and platform independent.
- ✓ AWT controls are part of **java.awt package**.
- ✓ Controls are added to a container (Frame, Panel, etc.).

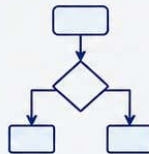
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101





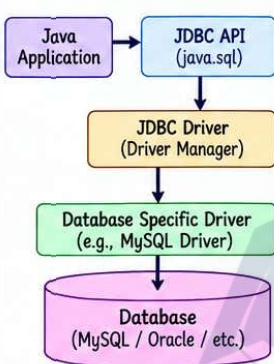
1. What is JDBC ?

- JDBC (**Java Database Connectivity**) is a Java API that provides a **standard interface** for connecting Java applications with **relational databases**.
- It allows Java programs to send SQL commands, execute them and process the results from the database.
- JDBC is **platform independent** and works with multiple databases like MySQL, Oracle, SQL Server, PostgreSQL etc.
- It is a part of the **java.sql** and **javax.sql** packages.
- JDBC uses **database drivers** to communicate with the database.

2. Need of JDBC

- To provide a **standard way** to access different databases.
- To make database programming easy in Java.
- To achieve **database independence** (write same code for different databases by using appropriate drivers).
- To support **enterprise applications** like web applications.
- To enable dynamic and real-time data access.
- It helps in building **database-driven applications** such as online banking, e-commerce systems etc.

3. Architecture of JDBC



Components :

- JDBC API**
Provides classes and interfaces (**java.sql** package).
- JDBC Driver Manager**
Manages the database drivers and establishes a connection.
- JDBC Driver**
Translates JDBC calls into database-specific protocols.
- Database**
The actual relational database (e.g., MySQL, Oracle, SQL Server).

4. JDBC Drivers

- A JDBC driver is a software component that enables communication between a Java application and a specific database.
- JDBC uses driver to translate Java calls into database-specific protocol.

Types of JDBC Drivers :

Type	Description	Example
Type 1 (JDBC-ODBC Bridge)	Converts JDBC calls into ODBC calls. (older, rarely used)	MS Access (ODBC)
Type 2 (Native-API)	Uses database vendor's native library.	Oracle
Type 3 (Network Protocol)	Converts JDBC calls into network protocol (independent of DB).	Middleware server
Type 4 (Thin Driver)	Directly converts JDBC calls into database protocol (most widely used).	MySQL, PostgreSQL, Oracle

5. Working of JDBC (Steps)

- Load and Register the JDBC Driver**
(Driver is loaded into memory).
- Establish a Connection**
(Create connection with database).
- Create a Statement**
(Used to execute SQL queries).
- Execute SQL Queries**
(Use `executeQuery()` or `executeUpdate()`).
- Process the Results**
(Use `ResultSet` for `SELECT` queries).
- Close the Resources**
(Close `ResultSet`, `Statement` and `Connection`).

6. Important JDBC Interfaces (java.sql package)

Interface	Description
Driver	Implemented by database vendors.
DriverManager	Manages the JDBC drivers.
Connection	Represents a connection with the database.
Statement	Used to execute static SQL queries.
PreparedStatement	Used to execute precompiled SQL queries (more efficient and secure).
CallableStatement	Used to execute stored procedures.
ResultSet	Represents the result of a <code>SELECT</code> query.
ResultSetMetaData	Provides metadata about the result set.
DatabaseMetaData	Provides metadata about the database.

7. Basic JDBC Program

```

import java.sql.*; // Import JDBC package
public class JDBCdemo {
    public static void main(String[] args) {
        try {
            // 1. Load and Register Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish Connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "password");

            // 3. Create Statement
            Statement stmt = con.createStatement();
            // 4. Execute Query
            ResultSet rs = stmt.executeQuery("select * from student");
            // 5. Process Result
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " " + rs.getString(2));
            }
            // 6. Close Resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
  
```

8. SQL Execution Methods

Method	Description
executeQuery()	Used for <code>SELECT</code> queries. Returns a <code>ResultSet</code> object.
executeUpdate()	Used for <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> queries. Returns number of rows affected.
execute()	Can be used for any type of SQL query.

9. Advantages of JDBC

- Platform independent (works on any OS).
- Supports multiple databases.
- Simple and easy to use.
- Powerful and flexible.
- Suitable for enterprise and web applications.
- Supports stored procedures.
- Provides better security using `PreparedStatement` (prevents SQL injection).

10. Applications of JDBC

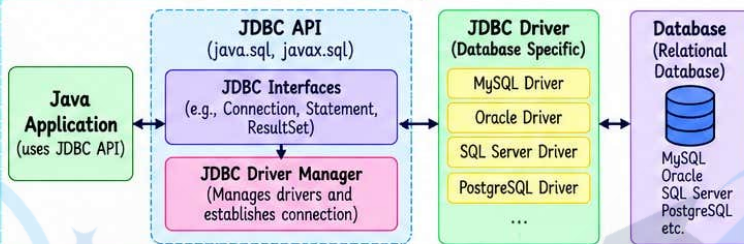




1. What is JDBC ?

- JDBC (**Java Database Connectivity**) is a **standard Java API** to connect Java applications with **relational databases**.
- It provides classes and interfaces to send SQL commands, execute them and process the results from the database.
- JDBC is **platform independent** and works with multiple databases like MySQL, Oracle, SQL Server, PostgreSQL etc.
- It is a part of Java Standard Edition (JSE) and belongs to **java.sql** and **javax.sql** packages.
- JDBC uses **database drivers** to communicate with the database.

2. JDBC Architecture (Overall View)



JDBC Architecture consists of the application, JDBC API, Driver Manager, database-specific driver, and the database. The JDBC API provides a standard interface, while the driver translates the JDBC calls into database-specific protocol.

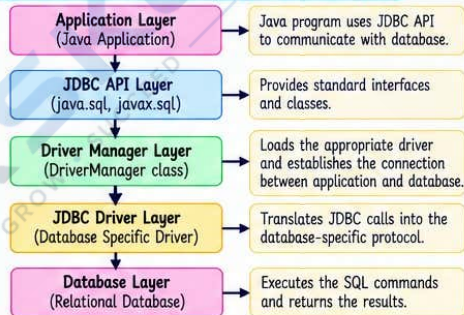
3. Components of JDBC Architecture

Component	Description
Java Application	Uses JDBC API to connect and communicate with the database.
JDBC API (java.sql, javax.sql)	Provides standard interfaces and classes for database access.
JDBC Interfaces	Define methods to connect, execute queries and process results (e.g., Connection, Statement, ResultSet).
JDBC Driver Manager	Manages database drivers and establishes the connection between application and the database.
JDBC Driver (Database Specific)	Translates JDBC calls into database-specific commands.
Database	Stores data and processes SQL commands (e.g., MySQL, Oracle, SQL Server, PostgreSQL).

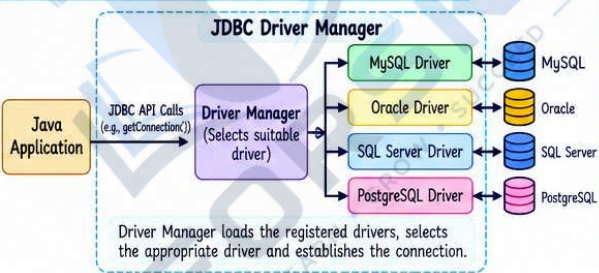
4. Working Flow of JDBC Architecture

- Java application **loads and registers** the JDBC driver (Class.forName()).
- Driver Manager identifies the suitable driver.
- Driver **establishes a connection** with the database (using getConnection()).
- Application creates Statement/PreparedStatement object.
- SQL queries are executed using **executeQuery()** or **executeUpdate()**.
- Database processes the SQL query.
- Result is returned in the form of **ResultSet** (for SELECT).
- Application processes the results.
- Finally, the resources are closed (ResultSet, Statement, Connection).

5. Detailed JDBC Architecture with Layers



6. Internal Architecture of JDBC Driver Manager



7. Advantages of JDBC Architecture

- Platform independent (works on any OS).
- Supports multiple databases.
- Provides a standard API.
- Database independence.
- Easy to develop database applications.
- Suitable for enterprise and web applications.
- Supports dynamic and real-time data access.
- Secure and efficient.

8. JDBC Architecture for Different Drivers

Driver Type	Class Name (Example)	Database
Type 1 (JDBC-ODBC Bridge)	sun.jdbc.odbc.JdbcOdbcDriver	Any (via ODBC)
Type 2 (Native-API)	oracle.jdbc.driver.OracleDriver	Oracle
Type 3 (Network Protocol)	com.mysql.cj.jdbc.Driver	MySQL (Middleware)
Type 4 (Thin Driver)	com.mysql.cj.jdbc.Driver	MySQL (Direct)

9. Example Flow (Connection to MySQL)

- Load and register the driver:**
Class.forName("com.mysql.cj.jdbc.Driver");
- Establish connection:**
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root", "password");
- Create statement:**
Statement stmt = con.createStatement();
- Execute query:**
ResultSet rs = stmt.executeQuery("select * from student");
- Process result:**
while(rs.next()) { System.out.println(rs.getInt(1)); }
- Close resources:**
rs.close(); stmt.close(); con.close();

10. Key Points

- JDBC API provides a standard interface.
- Driver Manager** manages drivers and establishes connections.
- Driver translates JDBC calls into database-specific commands.
- Different **types of drivers** are available (Type 1, 2, 3, 4).
- JDBC follows a **layered architecture**.
- It provides **database independence** and **platform independence**.



1. What is JDBC API ?

- JDBC API (**Application Programming Interface**) is a **standard** set of classes and interfaces provided by **Java** to connect and interact with relational databases.
- It is defined in the **java.sql** and **javax.sql** packages.
- It allows Java programs to send SQL commands (queries) to a database, execute them and process the results.
- JDBC API provides a **database independent** way of accessing different databases using the same set of interfaces and classes.

2. Package of JDBC API

- JDBC API is mainly available in two packages:

① java.sql

Contains all the core JDBC classes and interfaces.

② javax.sql

Contains server side classes for connection pooling, distributed transactions, etc.

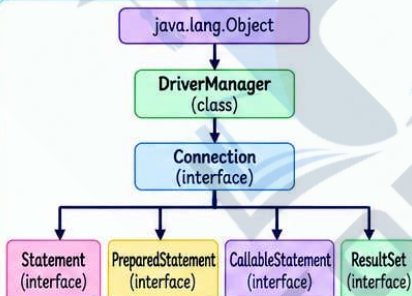
3. Purpose / Features of JDBC API

- ✓ Provides standard interfaces to connect with databases.
- ✓ Supports multiple databases.
- ✓ Database independent (same code works with different databases).
- ✓ Allows execution of SQL queries.
- ✓ Provides ways to process the results.
- ✓ Supports transaction management.
- ✓ Simple and easy to use.
- ✓ Suitable for developing database driven applications such as web applications, ERP systems, etc.

4. Main Components of JDBC API

Component / Interface	Description
DriverManager	Manages a list of database drivers and establishes connection to the database.
Connection	Represents a connection with the database.
Statement	Used to execute static SQL queries.
PreparedStatement	Used to execute precompiled SQL queries (more efficient and secure).
CallableStatement	Used to execute stored procedures.
ResultSet	Represents the result of a SELECT query.
ResultSetMetaData	Provides metadata about the result set.
DatabaseMetaData	Provides metadata about the database.
SQLException	Handles database related errors.

5. JDBC API Hierarchy



6. Important JDBC API Interfaces

Interface / Class	Description
DriverManager	Used to load/register the driver and establish the connection.
Connection	Represents a connection with the database.
Statement	Used to execute simple SQL queries.
PreparedStatement	Used to execute precompiled SQL queries with parameters.
CallableStatement	Used to execute stored procedures.
ResultSet	Represents the results of a query.
ResultSetMetaData	Provides information about the columns in the ResultSet.
DatabaseMetaData	Provides information about the database (e.g., tables, version, etc.).

7. Working of JDBC API (Steps)

1. Load and register the JDBC driver (DriverManager).
2. Establish a connection (using DriverManager.getConnection()).
3. Create a Statement / PreparedStatement object.
4. Execute SQL queries (executeQuery() / executeUpdate()).
5. Process the results (using ResultSet).
6. Close the resources (ResultSet, Statement, Connection).

8. Example Code using JDBC API

```

import java.sql.*;
public class JDBCdemo {
    public static void main(String[] args) {
        try {
            // 1. Load and register JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "password");
            // 3. Create Statement
            Statement stmt = con.createStatement();
            // 4. Execute Query
            ResultSet rs = stmt.executeQuery(
                "select * from student");
            // 5. Process Result
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " " +
                    rs.getString(2));
            }
            // 6. Close Resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println("Error: " + e);
        }
    }
}
  
```

9. Example: Using PreparedStatement

```

PreparedStatement ps = con.prepareStatement(
    "insert into student values (?, ?)");
ps.setInt(1, 101);
ps.setString(2, "Yunus");
int rows = ps.executeUpdate();
System.out.println(rows + " record inserted");
ps.close();
  
```

10. Example: Using CallableStatement

```

CallableStatement cs = con.prepareCall(
    "{call getStudent(?)}");
cs.setInt(1, 101);
ResultSet rs = cs.executeQuery();
while (rs.next()) {
    System.out.println(rs.getInt(1) + " " +
        rs.getString(2));
}
rs.close();
cs.close();
  
```

11. Key Points to Remember

- JDBC API is defined in **java.sql** and **javax.sql** packages.
- DriverManager** class is used to load the driver and get connection.
- Connection** is the session with database.
- Use **PreparedStatement** to prevent SQL injection.
- ResultSet** is used to process query results.
- Always close **ResultSet**, **Statement** and **Connection** objects after use.
- JDBC API is **database independent**.
- It provides standard interfaces for database access.

Types of JDBC Drivers



1. What are JDBC Drivers ?

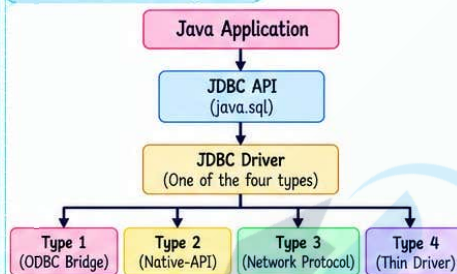
- JDBC Drivers are software components that enable Java applications to **communicate** with a **specific database**.
- They implement the JDBC interfaces and **translate Java method calls** into database-specific protocol and format understood by the database.
- Different types of JDBC drivers are available to connect with different databases.

2. Types of JDBC Drivers

JDBC provides four types of drivers:

- Type 1** – JDBC-ODBC Bridge Driver
- Type 2** – Native-API Driver
- Type 3** – Network Protocol Driver
- Type 4** – Thin Driver (JDBC Driver)

3. Overview Diagram

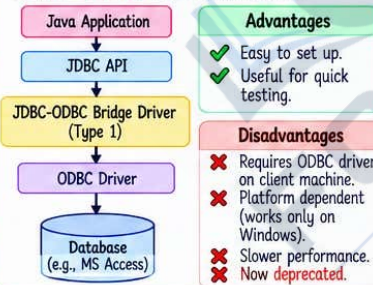


4. Key Points

- ✓ JDBC API is platform independent, but **driver implementation depends** on the database.
- ✓ Different types of drivers are designed for different environments and requirements.
- ✓ **Type 1 is now obsolete.**
- ✓ Type 2, Type 3 and Type 4 are commonly used.
- ✓ **Type 4 is the most widely used and recommended driver.**
- ✓ Driver translates Java method calls into database-specific protocol.

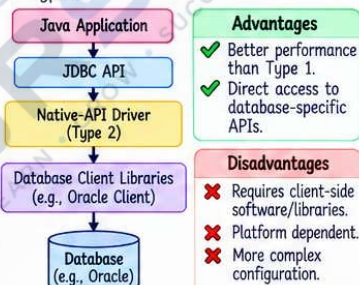
5. Type 1 : JDBC-ODBC Bridge Driver

- Converts JDBC calls into ODBC calls.
- Uses **ODBC driver**, which is installed on the client machine.
- This driver acts as a **bridge** between JDBC and ODBC.
- Suitable for **small and testing environments**.
- Now obsolete** (not recommended).



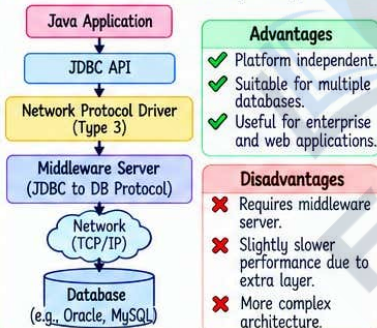
6. Type 2 : Native-API Driver

- Converts JDBC calls **directly** into database-specific **native API calls**.
- Uses client-side libraries of the database.
- Partially platform dependent (requires database client software).
- Provides **better performance** than Type 1.



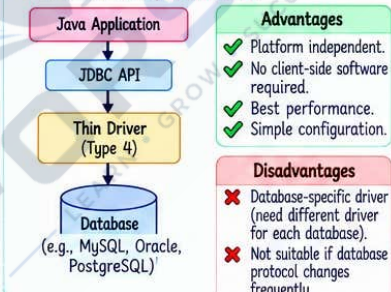
7. Type 3 : Network Protocol Driver

- Converts JDBC calls into a database-independent **network protocol**.
- Uses a **middleware server** which translates the requests to database-specific protocol.
- Completely platform **independent**.
- Suitable for web and enterprise applications.



8. Type 4 : Thin Driver (JDBC Driver)

- Converts JDBC calls directly into **database-specific protocol**.
- Does not require any client-side libraries or middleware server.
- Completely platform **independent**.
- Most widely used** driver in real-world applications.
- Provides **best performance**.



9. Comparison of JDBC Drivers

Type	Name	Translation	Client Software Required	Middleware Server	Platform Independence	Performance	Usage
Type 1	JDBC-ODBC Bridge	JDBC → ODBC → DB	Yes (ODBC driver)	No	No (Platform dependent)	Slow	Now obsolete
Type 2	Native-API Driver	JDBC → Native API → DB	Yes (Database client libraries)	No	Partially dependent	Better than Type 1	Limited use
Type 3	Network Protocol Driver	JDBC → Network Protocol → DB	No	Yes (Middleware server)	Yes (Platform independent)	Moderate	Enterprise / Web applications
Type 4	Thin Driver	JDBC → Database Protocol → DB	No	No	Yes (Platform independent)	Best	Most widely used



1. Install and Set Up Environment

- Install **JDK** (Java Development Kit).
- Set the path of JDK in **environment variables** (JAVA_HOME, Path).
- Download the **JDBC Driver** (MySQL/Oracle etc.).
- Add the JDBC driver JAR file to your project's **classpath** (in IDE or manually).

2. Import Required Packages

- Import the JDBC package in your Java program.

```
import java.sql.*;
```



java.sql package contains all the classes and interfaces required for JDBC programming.

3. Load and Register the JDBC Driver

- Load the database driver class using **Class.forName()** method.
- This tells Java to load and register the driver in memory.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- For different databases, use different driver class names.

Database	Driver Class Name
MySQL	com.mysql.cj.jdbc.Driver
Oracle	oracle.jdbc.driver.OracleDriver
PostgreSQL	org.postgresql.Driver

4. Establish a Connection

- Use **DriverManager.getConnection()** method to create a connection with the database.

```
Connection con =
DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/college",
    "root", "password");
```

JDBC URL Syntax (MySQL):

```
jdbc:mysql://hostname:port/database_name
• hostname: localhost (default)
• port: 3306 (default)
• database_name: name of the database
```

5. Create a Statement Object

- Use **Statement**, **PreparedStatement** or **CallableStatement** to execute SQL queries.
- Statement is used for simple queries.

```
Statement stmt = con.createStatement();
```



For dynamic or parameterized queries, use **PreparedStatement**.

6. Execute SQL Queries

- Use **executeQuery()** for SELECT queries (it returns a **ResultSet**).
- Use **executeUpdate()** for INSERT, UPDATE, DELETE queries (it returns the number of affected rows).

```
ResultSet rs = stmt.executeQuery(
    "select * from student");
int rows = stmt.executeUpdate(
    "insert into student values (1, 'Yunus')");
```

7. Process the Results

- For SELECT queries, the result is stored in a **ResultSet** object.
- Use **next()** method to move to the next row and get data using **getXXX()** methods.

```
while (rs.next()) {
    int id = rs.getInt(1);
    String name = rs.getString(2);
    System.out.println(id + " - " + name);
}
```

8. Close the Resources

- Always close **ResultSet**, **Statement** and **Connection** objects after use to release database resources.
- Use **close()** method.

```
rs.close();
stmt.close();
con.close();
```



It is good practice to close resources in reverse order of creation.

9. Complete Example Program (MySQL)

```
import java.sql.*;

public class JDBCdemo {
    public static void main(String[] args) {
        try {
            // 1. Load and Register Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish Connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college", "root", "password");
            // 3. Create Statement
            Statement stmt = con.createStatement();
            // 4. Execute Query
            ResultSet rs = stmt.executeQuery("select * from student");
            // 5. Process Results
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " - " + rs.getString(2));
            }
            // 6. Close Resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println("Error : " + e);
        }
    }
}
```

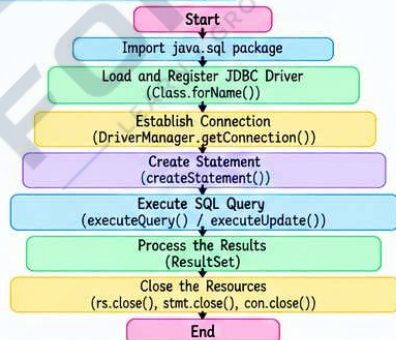
10. Output

```
1 - Ahmed
2 - Sara
3 - Imran
```

11. Important Points

- ✓ Import java.sql package.
- ✓ Load and register the driver.
- ✓ Use correct JDBC URL, username and password.
- ✓ Use appropriate statement object.
- ✓ Use **executeQuery()** or **executeUpdate()**.
- ✓ Process **ResultSet** using **next()** and **getXXX()**.
- ✓ Always close resources to avoid memory leaks.
- ✓ Handle exceptions using **try-catch**.

12. Flow Diagram of Steps

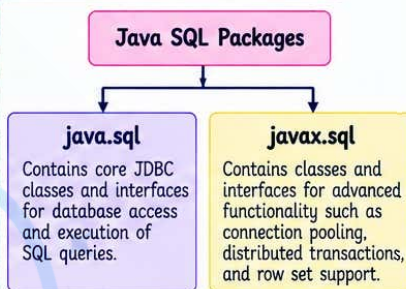




1. What are Java SQL Packages ?

- Java provides a **standard API** for database access through JDBC (Java Database Connectivity).
- The main packages related to JDBC are **java.sql** and **javax.sql**.
- These packages contain classes and interfaces required to connect to databases, execute SQL queries and process the results.
- They are included in the **JDK**, so no extra libraries are required for basic JDBC programming.

2. Java SQL Packages Overview



3. java.sql Package

- The **java.sql** package provides the core API for database access.
- It contains classes and interfaces to establish a connection, execute queries, process results and manage exceptions.
- It is the most commonly used package in JDBC programming.

Important Classes/Interfaces :

- DriverManager
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet
- ResultSetMetaData
- DatabaseMetaData
- SQLException
- Types

4. javax.sql Package

- The **javax.sql** package provides advanced features that are not available in **java.sql**.
- It is mainly used for **enterprise** applications (web applications, EJB, application servers) where connection pooling, distributed transactions, and row set support are required.
- It contains interfaces for managing database connections in a more efficient and scalable way.

Important Classes/Interfaces :

- DataSource
- ConnectionPoolDataSource
- PooledConnection
- XADataSource
- XAConnection
- RowSet
- CachedRowSet
- RowSetProvider

5. java.sql Package – Common Classes/Interfaces

Class / Interface	Description
DriverManager	Loads and registers the database driver and establishes connection.
Connection	Represents a connection with the database.
Statement	Used to execute simple SQL queries.
PreparedStatement	Used to execute parameterized SQL queries.
CallableStatement	Used to execute stored procedures.
ResultSet	Represents the result of a SELECT query.
ResultSetMetaData	Provides metadata about the result set.
DatabaseMetaData	Provides metadata about the database.
SQLException	Handles database-related errors.
Types	Defines SQL data types (e.g., VARCHAR, INTEGER).

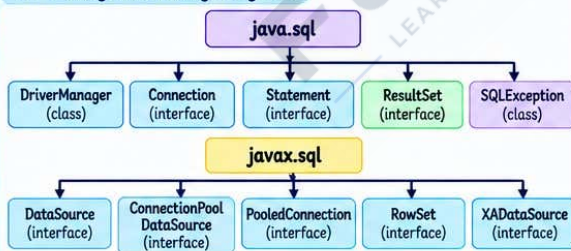
6. javax.sql Package – Common Classes/Interfaces

Class / Interface	Description
DataSource	Provides a factory for database connections (connection pooling).
ConnectionPoolDataSource	Manages a pool of connections.
PooledConnection	Represents a physical connection from the pool.
XADataSource	Supports distributed (transactional) databases.
XAConnection	Represents a connection that supports distributed transactions.
RowSet	Provides a disconnected, serializable row set.
CachedRowSet	A implementation of RowSet that can be cached and used offline.
RowSetProvider	Factory class to create RowSet objects.

7. Difference between java.sql and javax.sql

Feature	java.sql	javax.sql
Purpose	Core JDBC API for basic database access.	Provides advanced features for enterprise applications.
Main Use	Execute SQL queries, get results, handle exceptions.	Connection pooling, distributed transactions, RowSet support.
Common Classes	Connection, Statement, ResultSet, DriverManager, etc.	DataSource, ConnectionPoolDataSource, RowSet, etc.
Level	Basic JDBC functionality.	Advanced / Enterprise level.
Usage	Used in simple applications and standalone programs.	Used in web applications, application servers, and large enterprise systems.
Availability	Part of JDK (always available).	Part of JDK (available in javax.sql package).

8. Package Hierarchy Diagram



9. Example: Using Java SQL Package

```

import java.sql.*; // Import java.sql package
public class JDBCdemo {
    public static void main(String[] args) {
        try {
            // 1. Load and register driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college", "root", "password");
            // 3. Create statement
            Statement stmt = con.createStatement();
            // 4. Execute query
            ResultSet rs = stmt.executeQuery("select * from student");
            // 5. Process result
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " - " + rs.getString(2));
            }
            // 6. Close resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println("Error : " + e);
        }
    }
}
  
```

10. Key Points to Remember

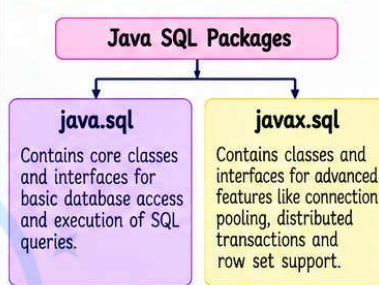
- ✓ **java.sql** provides core classes and interfaces for JDBC.
- ✓ **javax.sql** provides advanced features like connection pooling and RowSet.
- ✓ Both packages are part of JDK.
- ✓ Use **java.sql** for basic database operations.
- ✓ Use **javax.sql** for enterprise-level applications.
- ✓ Import the required package using **import java.sql.*;** or **import javax.sql.*;**
- ✓ **DriverManager** is in **java.sql** package.
- ✓ **DataSource** and connection pooling classes are in **javax.sql** package.
- ✓ These packages help in making database applications portable and platform independent.



1. What are Java SQL Packages ?

- Java provides a **standard API** for database access called JDBC (Java Database Connectivity).
- The Java SQL packages are **java.sql** and **javax.sql**.
- These packages contain a set of classes and interfaces that are used to connect with a database, execute SQL statements, process the results and manage exceptions.
- They are included in the **JDK**, so no additional libraries are required for basic JDBC programming.

2. Java SQL Packages Overview



3. Purpose of Each Package

java.sql package

- Provides the core JDBC API.
- Includes essential classes and interfaces for database connectivity.
- Used to execute SQL queries, process results and handle exceptions.
- Sufficient for simple and standalone Java applications.

javax.sql package

- Provides extended JDBC API.
- Used for enterprise-level applications.
- Includes classes for connection pooling, distributed transactions and row set support.
- Works in conjunction with java.sql package.

4. Key Classes/Interfaces

java.sql (Core JDBC)

- DriverManager
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet
- ResultSetMetaData
- DatabaseMetaData
- SQLWarning
- SQLException
- Ref
- Blob / Clob
- Types

javax.sql (Advanced/Enterprise)

- DataSource
- ConnectionPoolDataSource
- PooledConnection
- XADataSource
- XAConnection
- RowSet
- CachedRowSet
- RowSetProvider

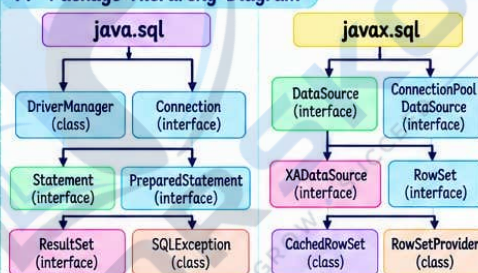
5. Common Classes in java.sql Package

Class / Interface	Description
DriverManager	Loads and registers the database driver and establishes connection.
Connection	Represents a connection with the database.
Statement	Used to execute simple SQL queries.
PreparedStatement	Used to execute parameterized SQL queries (more efficient and secure).
CallableStatement	Used to execute stored procedures.
ResultSet	Represents the result of a SELECT query.
ResultSetMetaData	Provides metadata about the result set.
DatabaseMetaData	Provides metadata about the database.
SQLException	Handles database related errors.
Types	Defines SQL data types (e.g. INTEGER, VARCHAR).

6. Common Classes in javax.sql Package

Class / Interface	Description
DataSource	Provides a factory for obtaining database connections (used for connection pooling).
ConnectionPoolDataSource	Manages a pool of database connections.
PooledConnection	Represents a physical connection from the pool.
XADataSource	Provides support for distributed (transaction) databases.
XAConnection	Represents a connection that supports distributed transactions (XA).
RowSet	Represents a disconnected, scrollable result set.
CachedRowSet	Implementation of RowSet that can be cached and used offline.
RowSetProvider	Factory class to create RowSet objects.

7. Package Hierarchy Diagram



8. Example: Using java.sql Package (Basic JDBC Program)

```

import java.sql.*;
public class JDBCdemo {
    public static void main(String[] args){
        try {
            // 1. Load and register driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college", "root", "password");
            // 3. Create statement
            Statement stmt = con.createStatement();
            // 4. Execute query
            ResultSet rs = stmt.executeQuery("select * from student");
            // 5. Process result
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " - " + rs.getString(2));
            }
            // 6. Close resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println("Error : " + e);
        }
    }
}
  
```

9. Example: Using javax.sql Package (DataSource)

```

import java.sql.*;
import javax.sql.*;
public class DataSourceDemo {
    public static void main(String[] args) {
        try {
            // 1. Create DataSource object
            MySQLDataSource ds = new MySQLDataSource();
            ds.setURL("jdbc:mysql://localhost:3306/college");
            ds.setUser("root");
            ds.setPassword("password");
            // 2. Get connection from DataSource
            Connection con = ds.getConnection();
            // 3. Execute query
            PreparedStatement ps = con.prepareStatement(
                "select * from student where id = ?");
            ps.setInt(1, 1);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                System.out.println(rs.getInt(1) + " - " + rs.getString(2));
            }
            // 4. Close resources
            ps.close();
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
  
```

10. Important Points

- ✓ Java SQL packages are part of the **standard JDK**.
- ✓ **java.sql** provides the core JDBC API.
- ✓ **javax.sql** provides advanced features for enterprise applications.
- ✓ Both packages work together.
- ✓ Use **java.sql** for simple applications.
- ✓ Use **javax.sql** for connection pooling and distributed transactions.
- ✓ These packages make database programming platform independent.
- ✓ No extra libraries are required for basic JDBC programs.
- ✓ Classes and interfaces in these packages help in executing SQL queries, processing results and handling exceptions.
- ✓ Widely used in real-world applications (web apps, desktop apps, ERP).



1. INSERT Operation (Concept)

- INSERT statement is used to add new records into a table.
- In JDBC, it is executed using the `executeUpdate()` method of Statement, PreparedStatement or CallableStatement.
- It returns the number of rows affected.
- PreparedStatement is preferred for inserting data because it is more secure and can handle parameters.

2. Syntax (INSERT using PreparedStatement)

```
String sql = "INSERT INTO table_name
              (column1, column2, ...)
              VALUES (?, ?, ...)";
```

```
PreparedStatement pst = con.prepareStatement(sql);
pst.setXXX(1, value1);
pst.setXXX(2, value2);
...
int rows = pst.executeUpdate();
```

setXXX() methods:

- `setInt()`, `setString()`, `setDouble()`,
- `setDate()`, `setBoolean()` etc. based on data type.

3. Example: Inserting a Record

```
import java.sql.*;
public class InsertDemo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "password");
            String sql = "INSERT INTO student(id, name, age)
                        + " VALUES (?, ?, ?)";
            PreparedStatement pst = con.prepareStatement(sql);
            pst.setInt(1, 101);
            pst.setString(2, "Ayesha");
            pst.setInt(3, 20);
            int rows = pst.executeUpdate();
            System.out.println(rows + " record inserted");
            con.close();
        } catch (Exception e) {
            System.out.println("Error: " + e);
        }
    }
}
```

4. Sample Table (Before & After Insert)

Before Inserting:

ID	Name	Age
101	Ahmed	19
102	Sara	21



After Inserting:

ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Ayesha	20

5. UPDATE Operation (Concept)

- UPDATE statement is used to modify existing records in a table.
- In JDBC, it is executed using the `executeUpdate()` method.
- It returns the number of rows affected.
- PreparedStatement is recommended to prevent SQL injection.

6. Syntax (UPDATE using PreparedStatement)

```
String sql = "UPDATE table_name
              SET column1 = ?, column2 = ?
              WHERE condition";
```

```
PreparedStatement pst = con.prepareStatement(sql);
pst.setXXX(1, value1);
pst.setXXX(2, value2);
pst.setXXX(3, conditionValue);
int rows = pst.executeUpdate();
```

Note:

- The WHERE clause is important to specify which record to update.
- Without WHERE clause, all records will be updated.

7. Example: Updating a Record

```
import java.sql.*;
public class UpdateDemo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "password");
            String sql = "UPDATE student SET age = ?
                        WHERE id = ?";
            PreparedStatement pst = con.prepareStatement(sql);
            pst.setInt(1, 21); // new age
            pst.setInt(2, 103); // id of record to update
            int rows = pst.executeUpdate();
            System.out.println(rows + " record updated");
            con.close();
        } catch (Exception e) {
            System.out.println("Error: " + e);
        }
    }
}
```

8. Table After Update

Before Update:

ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Ayesha	20



After Update (Age changed to 21):

ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Ayesha	21

9. Output (Console)

Insert Output

1 record inserted

Update Output

1 record updated

10. Key Points to Remember

- ✓ INSERT and UPDATE are executed using `executeUpdate()`.
- ✓ PreparedStatement is preferred.
- ✓ Use appropriate `setXXX()` method based on data type.
- ✓ Always use WHERE clause in UPDATE.
- ✓ `executeUpdate()` returns the number of rows affected.
- ✓ Close Connection, PreparedStatement and other resources.
- ✓ Handle exceptions using `try-catch`.

11. Advantages of PreparedStatement

- ✓ Prevents SQL injection.
- ✓ Can reuse the same SQL statement.
- ✓ More efficient and faster for multiple insert/update operations.
- ✓ Supports parameterized queries.
- ✓ Better performance and security.



1. Introduction

- In JDBC, records in a database can be **selected** (retrieved), **inserted** (added), **updated** (modified) and **deleted**.
- These operations are performed using SQL queries with **Statement**, **PreparedStatement** or **CallableStatement**.
- PreparedStatement** is preferred because it is more secure, efficient and supports parameterized queries.

2. Selecting Records (SELECT)

- Used to retrieve data from a table.

Syntax:

```
String sql = "SELECT column1, column2
FROM table_name WHERE condition";
```

```
import java.sql.*;
public class SelectDemo {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "password");
        String sql = "SELECT * FROM student";
        PreparedStatement pst = con.prepareStatement(sql);
        ResultSet rs = pst.executeQuery();
        System.out.println("ID Name Age");
        while(rs.next()){
            System.out.println(rs.getInt("id") + " " +
                rs.getString("name") + " " + rs.getInt("age"));
        }
        rs.close(); pst.close(); con.close();
    }
}
```

3. Inserting Records (INSERT)

- Used to add new records in a table.

Syntax:

```
String sql = "INSERT INTO table_name
(column1, column2, ...) VALUES (?, ?, ...)";
```

```
String sql = "INSERT INTO student
(name, age) VALUES (?, ?)";
PreparedStatement pst = con.prepareStatement(sql);
pst.setString(1, "Ayesha");
pst.setInt(2, 20);
int rows = pst.executeUpdate();
System.out.println(rows + " record inserted");
```

4. Updating Records (UPDATE)

- Used to modify existing records.

Syntax:

```
String sql = "UPDATE table_name
SET column1 = ?, column2 = ? WHERE condition";
```

```
String sql = "UPDATE student
SET age = ? WHERE id = ?";
PreparedStatement pst = con.prepareStatement(sql);
pst.setInt(1, 21); // new age
pst.setInt(2, 103); // id of record to update
int rows = pst.executeUpdate();
System.out.println(rows + " record updated");
```

5. Deleting Records (DELETE)

- Used to remove records from a table.

Syntax: String sql = "DELETE FROM table_name WHERE condition";

```
String sql = "DELETE FROM student
WHERE id = ?";
PreparedStatement pst = con.prepareStatement(sql);
pst.setInt(1, 102); // id to delete
int rows = pst.executeUpdate();
System.out.println(rows + " record deleted");
```

7. Output (Console)

```

ID Name Age
101 Ahmed 19
103 Ayesha 21
104 Imran 20
1 record inserted
1 record updated
1 record deleted
```

9. Key Points

- ✓ SELECT is used to retrieve records.
- ✓ INSERT is used to add new records.
- ✓ UPDATE is used to modify existing records.
- ✓ DELETE is used to remove records.
- ✓ `executeQuery()` is used for SELECT.
- ✓ `executeUpdate()` is used for INSERT, UPDATE and DELETE.
- ✓ `PreparedStatement` is preferred for security and parameters, `Statement` for Prepared and Connection.

6. Example: Complete Program

Demonstrates selecting, inserting, updating and deleting.

```
import java.sql.*;
public class RecordOperations {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "password");

        // Insert
        PreparedStatement pst1 = con.prepareStatement(
            "INSERT INTO student(name,age) VALUES(?,?)");
        pst1.setString(1, "Ayesha"); pst1.setInt(2, 20);
        pst1.executeUpdate();

        // Update
        PreparedStatement pst2 = con.prepareStatement(
            "UPDATE student SET age=? WHERE id=?");
        pst2.setInt(1, 21); pst2.setInt(2, 103);
        pst2.executeUpdate();

        // Delete
        PreparedStatement pst3 = con.prepareStatement(
            "DELETE FROM student WHERE id=?");
        pst3.setInt(1, 102); pst3.executeUpdate();

        // Select and display
        PreparedStatement pst4 = con.prepareStatement("SELECT * FROM student");
        ResultSet rs = pst4.executeQuery();
        System.out.println("ID Name Age");
        while(rs.next()){
            System.out.println(rs.getInt("id") + " " + rs.getString("name") + " " + rs.getInt("age"));
        }
        rs.close(); pst1.close(); pst2.close(); pst3.close(); pst4.close(); con.close();
    }
}
```

8. Table Records at Each Step (Example)

Initial Table		
ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Imran	20

After INSERT (Ayesha, 20)

ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Imran	20
104	Ayesha	20

After UPDATE (ID 103, Age = 21)

ID	Name	Age
101	Ahmed	19
102	Sara	21
103	Imran	21
104	Ayesha	20

After DELETE (ID 102)

ID	Name	Age
101	Ahmed	19
103	Imran	21
104	Ayesha	20



The tables show how records change step by step when we perform INSERT, UPDATE and DELETE operations.

10. Summary of JDBC Record Operations

Operation	SQL Command	JDBC Method	Purpose
Select	SELECT	<code>executeQuery()</code>	Retrieve records (ResultSet)
Insert	INSERT	<code>executeUpdate()</code>	Add new records
Update	UPDATE	<code>executeUpdate()</code>	Modify existing records
Delete	DELETE	<code>executeUpdate()</code>	Remove records